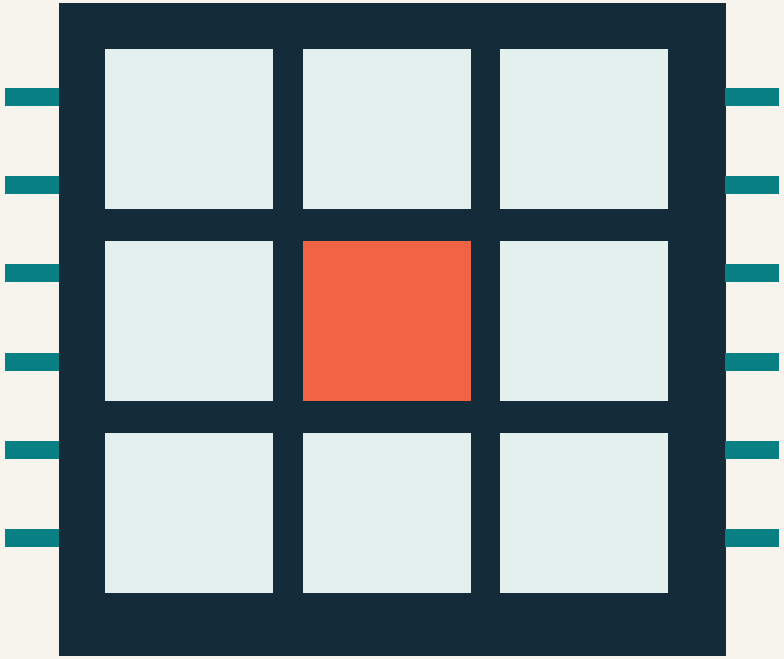
Your first Tiny Tapeout chip

Understand it.
Build it. Test it.

One real 8-bit counter, from your first bit to a checked chip layout.



Illustrative shared chip

What you will understand and produce

1. Predict an eight-bit counter: reset, count, hold and wrap.
2. Explain its Verilog, fixed chip interface and Python checks.
3. Create a project and inspect the real automated results.
4. Keep the source, datasheet, waveform and checked layout together.

WORKED EXAMPLE

One circuit.
0 → 1 → 2 … 255 → 0

One checked revision.
Several kinds of evidence.

CHECK You can name both the circuit behaviour and the files the exercise produces.

Your route through the course

01 Foundations
Slides 9–33
Bits, logic, stored state and clock edges

02 Read the code
Slides 34–68
Counter, wrapper and self-checking test

03 Browser lab
Slides 69–127
Template, files, workflows and real waves

04 Physical design
Slides 128–151
Cells, wires, timing and layout checks

05 Fabrication
Slides 152–161
Free endpoint, later order and silicon

06 Practise
Slides 162–195
Optional local setup and worked exercises

Get your learning workspace ready

1. Use a laptop, browser, internet access and your own GitHub account.
2. Keep counter-starter.zip and the Word manual beside this deck.
3. Create tiny-tapeout-learning and an evidence folder for downloads.
4. Use a public practice repository and standard Actions runners.

CHECK You have the ZIP, manual, account and a place to save your evidence.

Understand the free stopping point

DO IN THIS COURSE
Write and test the circuit. Generate a layout, inspect the checks and archive matching artifacts.

LATER MANUFACTURING
Choose an approved funded route, create an order and explicitly submit an accepted revision.

CHECK A generated layout is a file; ordering and submitting for manufacture are separate actions.

Read each step as a small experiment

1. WHY: understand what the step achieves.
2. DO: follow the action or predict the example.
3. CHECK: look for observable evidence of success.
4. If it fails, identify the first useful error before continuing.

CHECK You can explain what a step should produce before doing it.

Keep the example and your version distinct

REFERENCE
Repository: tt-sky130-counter
Branch: worked-example
Verified commit: 61ef06b

YOUR PROJECT
Your account and repository
Your own saved commit
Your runs must test that same commit

CHECK Each result you trust belongs to the exact source version you intend to review.

Use the seven-file starter correctly

Official template

Workflows, dependencies and process settings

Seven-file ZIP

Counter RTL, wrapper, test and datasheet text

Your saved project

A complete example on one reviewed commit

CHECK The ZIP supplements the template; preserve both the paths and template infrastructure.

Our first chip circuit counts from 0 to 255

1. An ASIC is a chip built for a particular job.
2. Our job: remember an 8-bit count and update it on a clock edge.
3. RTL (register-transfer level) describes the registers and logic.

Reset → 0; enabled rising edges → 1, 2, 3 … 255, 0.

CHECK What must the circuit remember between clock edges? The current count.

Verilog describes connected hardware

Software instructions run on an existing processor.

An HDL (hardware description language) describes circuits that can operate together.

The clocked count and continuous wrap logic coexist.

count remembers a value; wrap continuously examines count and en.

CHECK Does the chip execute a Verilog text file? No: the file describes hardware.

Logic calculates. A register remembers.

Combinational logic calculates from its current inputs.

A register stores a value between active clock edges.

Our count is stored; our wrap flag is calculated.

At count = 255: en changes 1 → 0; wrap becomes 0, count stays 255.

CHECK Which output can change between rising edges here? wrap.

One bit has two logical values

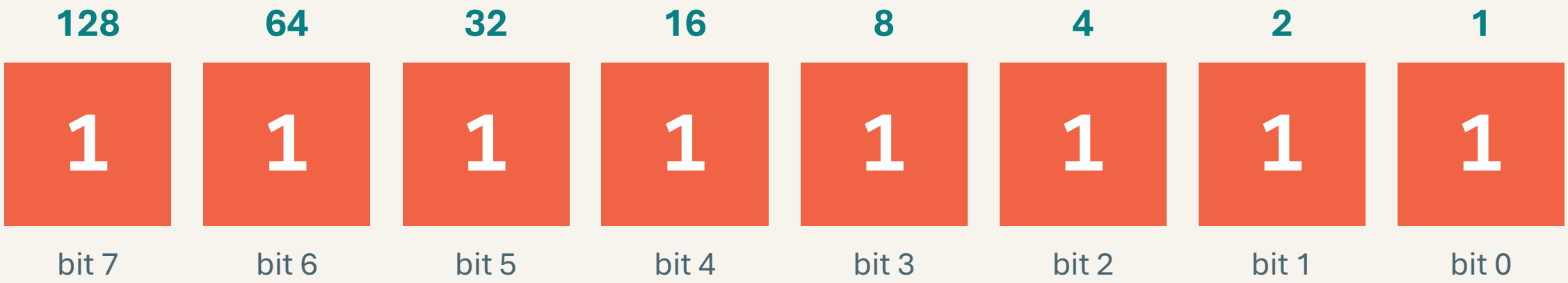
0	1
Logical low	Logical high

ui_in[0] = 1 means the enable input is logically high.

A bit represents logical 0 or logical 1. • Physical pins use low and high voltage ranges. • Logical 1 does not mean exactly one volt.

CHECK Do not choose a supply voltage from the digit 1.

Eight bits give 256 patterns

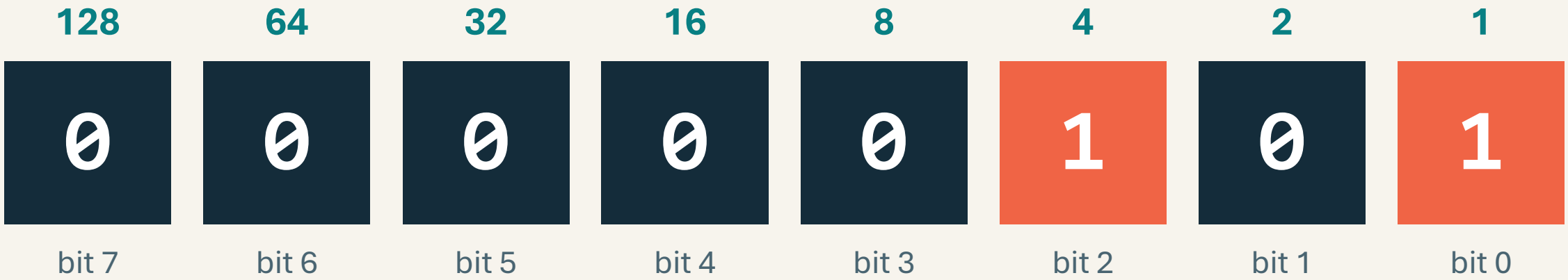


3 bits: 8 patterns, values 0–7. 8 bits: 256 patterns, values 0–255.

Each of eight bits has two possibilities: $2^8 = 256$. • Our unsigned values run from 0 through 255. • 00000000 is 0; 11111111 is 255.

CHECK 256 patterns does not mean the maximum count is 256.

Each position has a binary weight

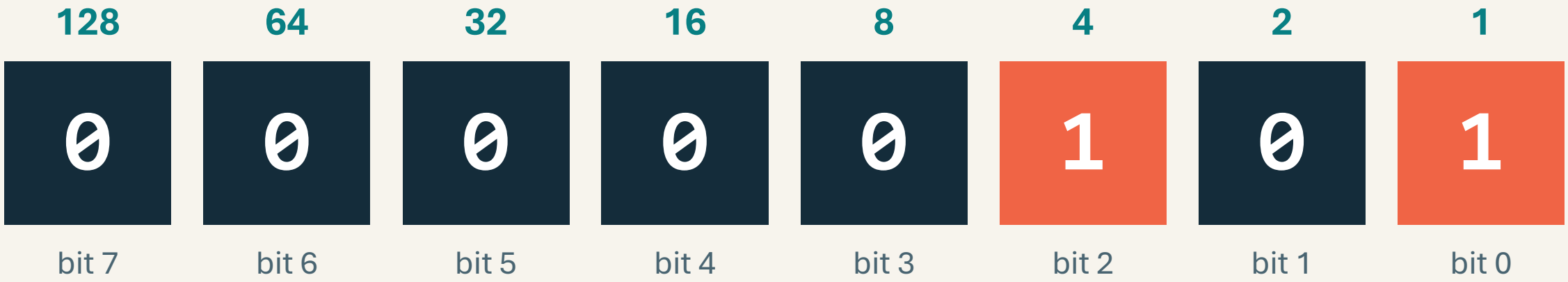


00000101 = 4 + 1 = 5.

From bit 7 to bit 0: 128, 64, 32, 16, 8, 4, 2, 1. • Add the weights whose bits are 1. • Bit 0 is the least significant bit.

CHECK For [7:0], count both endpoints: there are eight bits.

Count 5 appears on two output bits



Count 5 → uo_out = 00000101. A decimal display needs a decoder.

uo_out carries the count as eight binary output signals. • At count 5, uo_out[2] and uo_out[0] are high. • The other six output bits are low.

CHECK What is uo_out[1] at count 5? It is 0.

Try it: which pins represent 10?

TRY IT FIRST

Use the weights 128, 64, 32, 16, 8, 4, 2, 1.
Choose weights that add to decimal 10.
Name the output positions that are high.

What is uo_out when the unsigned count is 10?

WORKED ANSWER

00001010: uo_out[3] and uo_out[1] are high.

Three notations can describe the same value

8'b00000101: eight bits written in binary.

8'd5: eight bits written in decimal.

8'h05: eight bits written in hexadecimal.

8'b00000101 = 8'd5 = 8'h05; underscores do not add bits.

CHECK Read 1'b1 as a one-bit binary one, not a variable name.

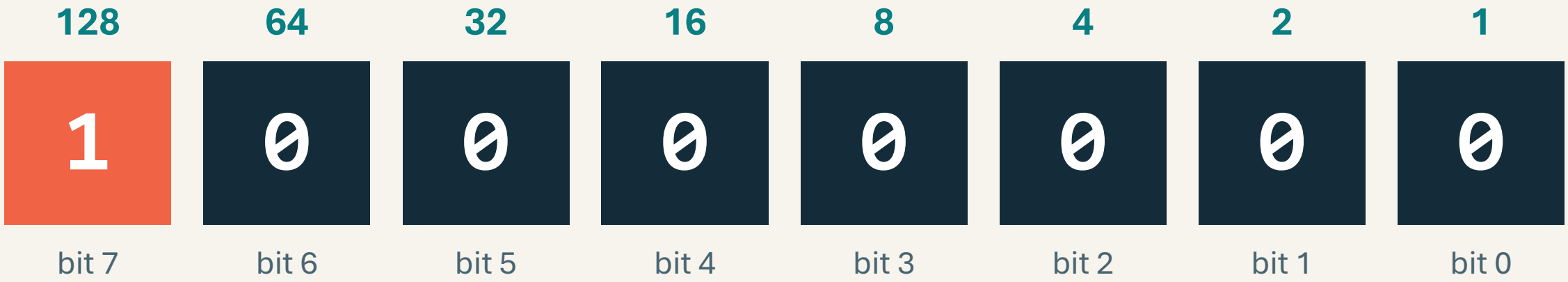
Keep the Tiny Tapeout interface intact

1. ui_in: eight dedicated input signals.
2. uo_out: eight dedicated output signals.
3. uio: eight bidirectional pins with three internal paths.
4. clk, rst_n and ena are control signals.

ui_in[0] → enable; uo_out[7:0] → count; uio[0] → wrap.

CHECK The three uio buses describe eight shared pins, not 24 pins.

Only input bit 0 enables this counter



ui_in = 128 → 10000000 → ui_in[0] = 0 → hold.

The wrapper connects ui_in[0] to the counter's en input. • 00000001 enables counting; 10000000 does not. • Other ui_in bits do not change this example's behaviour.

CHECK A nonzero input bus is not sufficient; inspect bit 0.

ena selects the project; en controls counting

ena belongs to the shared project-selection interface.

en belongs to our counter and comes from ui_in[0].

The test selects the project, then varies the counting enable.

Selected project: ena = 1. Hold counting: ui_in[0] = 0.

CHECK Do not replace the connection .en(ui_in[0]) with ena.

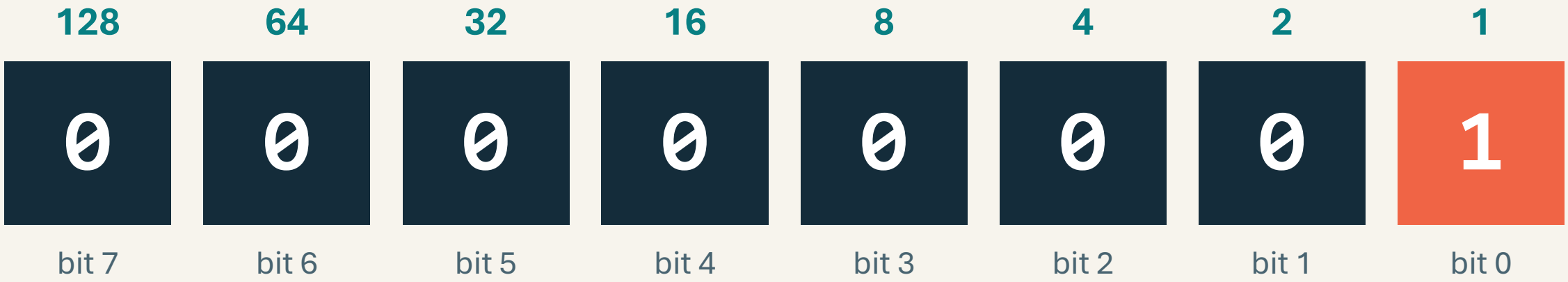
A bidirectional pin needs a direction

1. uio_in reads the pin’s input path.
2. uio_out supplies the pin’s output data.
3. uio_oe chooses direction: 1 = output, 0 = input.

For pin uio[0]: output data = wrap; output enable = 1.

CHECK uio_out = 0 and uio_oe = 0 means an inactive output driver.

00000001 makes only uio[0] an output



Direction mask 00000001 → one output driver, at uio[0].

The wrapper assigns uio_oe = 8'b0000_0001. • Bit 0 is an output; bits 7 through 1 remain inputs. • The direction pattern stays constant during this test.

CHECK Does uio_oe = 1 enable every pin? No, only bit 0.

Write the next-state rule before the code

1. At a rising edge: reset first, then enable, then hold.
2. Reset sets count to 0; enable adds 1 modulo 256.
3. With neither condition active, keep the current count.

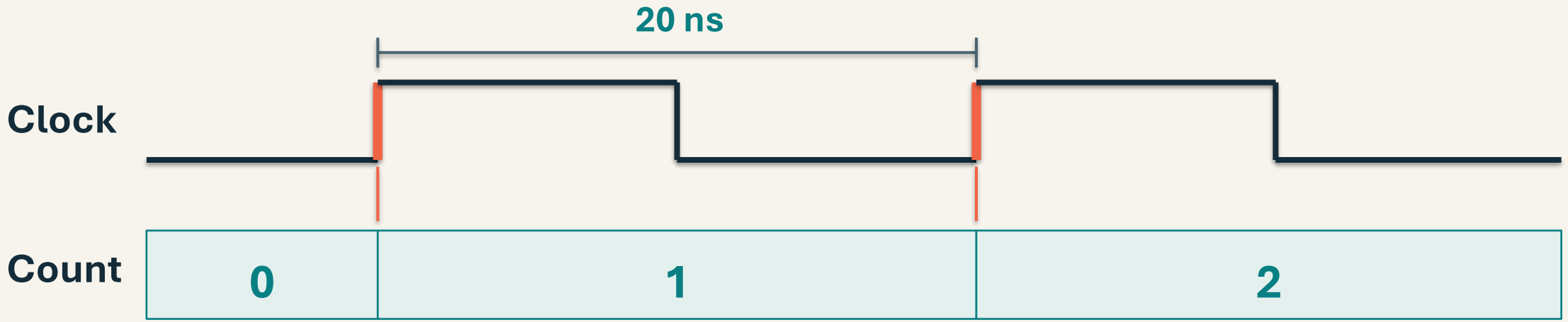
WORKED EXAMPLE

Before = 17; reset active and en = 1 → after the rising edge = 0.

CHECK A specification should let you predict an answer before running a tool.

A clock edge is the update event

A rising edge is 0 → 1; a held-high clock adds no edges.
20 ns per cycle = 50 MHz in this test.



Enable = 1; reset released. Count changes only at ↑clk.

Predicted RTL trace — not a recorded FST screenshot

CHECK A clock held high produces no further rising edges.

Follow six rising edges

Edge	rst_n	enable	Before	After	wrap
1	1	1	0	1	0
2	1	1	1	2	0
3	1	0	2	2	0
4	0	1	2	0	0
5	1	1	0	1	0
6	1	1	1	2	0

Predicted sequence: 0 → 1 → 2 → 2 → 0 → 1 → 2.

CHECK Reset is demonstrated at edge 4; hold is demonstrated at edge 3.

rst_n is active low; rst is active high

At the wrapper, rst_n = 0 means reset is asserted.

Inside the counter, rst = 1 means reset is asserted.

The wrapper inverts the signal: rst = ~rst_n.

rst_n 0 → rst 1 → reset requested. rst_n 1 → rst 0 → released.

CHECK A name documents intent; the inversion and clocked logic implement it.

This reset waits for the next rising edge

Edge	rst_n	enable	Before	After	wrap
next ↑	0	0	7	0	0

Between edges: count = 7. rst_n falls. Next ↑clk: count becomes 0.

CHECK Active-low does not automatically mean asynchronous.

Reset has priority over enable

Edge	rst_n	enable	Before	After	wrap
1	0	1	17	0	0
2	1	1	0	1	0

17 → reset + enable → 0; release reset + enable → 1.

CHECK Both controls active means reset, not reset followed by increment.

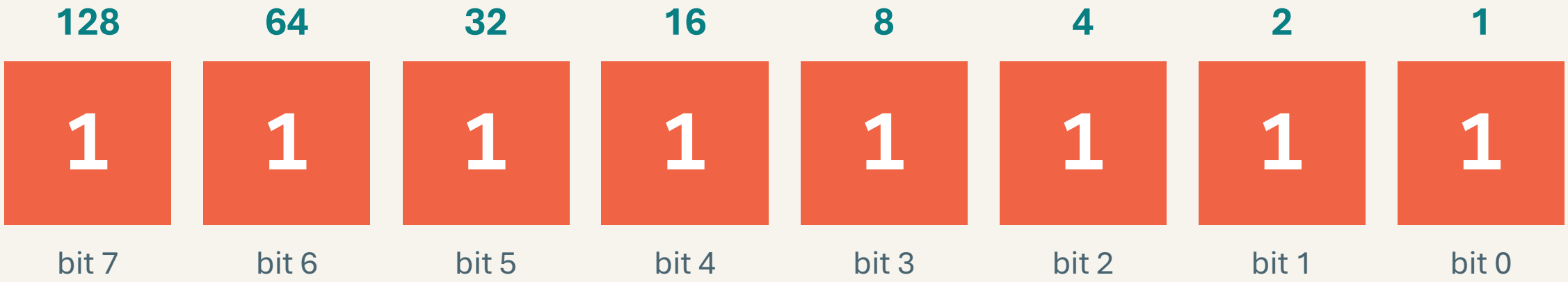
Hold the value while the clock continues

Edge	rst_n	enable	Before	After	wrap
1	1	0	10	10	0
2	1	0	10	10	0
3	1	1	10	11	0

Two disabled edges hold 10; re-enable at the next edge → 11.

CHECK With en = 0, can an active reset still clear count? Yes, at ↑clk.

255 + 1 becomes 0 in eight stored bits



11111111 + 1 = 1_00000000 → keep eight low bits → 00000000.

11111111 + 1 produces the nine-bit result 100000000. • Only the lowest eight bits fit in count.

- The stored result is 00000000: modulo-256 arithmetic.

CHECK Overflow wraps the count; it does not saturate at 255.

wrap is high at 255, before overflow

Edge	rst_n	enable	Before	After	wrap
1	1	1	253	254	0
2	1	1	254	255	1
3	1	1	255	0	0
4	1	1	0	1	0

Enabled sequence: 254 / wrap 0 → 255 / wrap 1 → 0 / wrap 0.

CHECK After the overflow edge, count = 0 and wrap = 0.

Disabling at 255 suppresses wrap

Edge	rst_n	enable	Before	After	wrap
1	1	0	255	255	0
2	1	0	255	255	0
3	1	1	255	0	0

At 255: en 1 → 0 clears wrap without changing the stored count.

CHECK Rows show wrap after each edge; re-enabling makes it high before overflow.

Reset does not directly mask the flag

The wrap expression uses en and count, not reset.

At count 255 with en = 1, lowering rst_n can leave wrap high.

At the next rising edge, reset stores 0 and wrap falls.

Before ↑clk: count 255, en 1, rst_n 0 → wrap may still be 1.

CHECK After that reset edge: count 0 and wrap 0. Polarity alone is insufficient.

A file, a module and an instance are different

1. src/counter.v is a file containing module counter.
2. src/project.v contains the Tiny Tapeout wrapper module.
3. counter_inst is one instance of counter inside that wrapper.

File project.v → module tt_um_mdkamrulislam_counter → instance counter_inst.

CHECK The top module is not named project merely because its file is project.v.

The top-module name must agree in three places

1. src/project.v: the module declaration.
2. info.yaml: project.top_module.
3. test/tb.v: the instantiated module type.

Exact shared name: tt_um_mdkamrulislam_counter.

CHECK Compare spelling and case in all three locations before committing.

Both build routes need both source files

```
# info.yaml excerpt
project:
  source_files:
    - "project.v"
    - "counter.v"

# test/Makefile excerpt
PROJECT_SOURCES = project.v counter.v
```

1. info.yaml lists the files for the layout build.
2. test/Makefile lists the files for RTL simulation.
3. Both must include project.v and counter.v.

CHECK A new RTL file needs consideration in both source lists.

File directives prevent hidden assumptions

```
`timescale 1ns / 1ps
`default_nettype none

// End of the counter file:
`default_nettype wire
```

1. timescale gives simulation time units and precision.
2. default_nettype none prevents accidental implicit wires.
3. The file restores default_nettype wire at its end.

CHECK timescale is a simulation convention, not a hardware clock source.

WIDTH makes the counter reusable

```
module counter #(
    parameter WIDTH = 8
) (
```

1. parameter WIDTH = 8 supplies the default width.
2. The wrapper explicitly chooses WIDTH = 8.
3. [WIDTH-1:0] becomes [7:0] for this instance.

WIDTH = 8 → [WIDTH-1:0] = [7:0] → eight positions.

CHECK This is a declaration excerpt; the ports follow in the real file.

Read each counter port by direction and width

```
input  wire             clk,
input  wire             rst,
input  wire             en,
output reg  [WIDTH-1:0] count,
output wire             wrap
```

1. clk, rst and en are one-bit input connections.
2. count has WIDTH bits and is assigned in a clocked block.
3. wrap is a one-bit output driven by continuous logic.

CHECK reg is a procedural type; the clocked assignment establishes stored state.

posedge clk tells us when to update

```
always @(posedge clk) begin
    if (rst)
        count <= {WIDTH{1'b0}};
    else if (en)
        count <= count + 1'b1;
end
```

1. always starts a hardware process.
2. @(posedge clk) waits for a rising clock transition.
3. begin and end group the statements in this process.

CHECK The event control contains clk only: reset is synchronous.

Reset writes WIDTH zeros into the register

```
if (rst)
    count <= {WIDTH{1'b0}};
```

1. 1'b0 is a one-bit binary zero.
2. {WIDTH{1'b0}} repeats that zero WIDTH times.
3. <= schedules the register update; it is an assignment here.

WIDTH = 8 → {WIDTH{1'b0}} = 00000000.

CHECK The if condition uses internal active-high rst, not external rst_n.

Enable adds one; no assignment means hold

```
else if (en)
    count <= count + 1'b1;
```

1. else if makes this branch conditional on reset being inactive.
2. count + 1'b1 calculates the next unsigned count.
3. If en is 0, this clocked block retains the stored value.

Before = 2. en 1 → after = 3. en 0 → after = 2.

CHECK Holding the register is not the same as assigning it zero.

Predict the branch before running anything

TRY IT FIRST

Begin at count 9; evaluate each rising edge in order.
A: reset off, enable on. B: reset off, enable off.
C: reset on, enable on. D: reset off, enable on.

Starting at 9, what values follow edges A, B, C and D?

WORKED ANSWER

10, 10, 0, 1. Reset wins at C; the disabled B edge holds 10.

Reduction AND detects all count bits high

```
assign wrap = en & (&count);
```

1. &count reduces eight bits to one result.
2. That result is 1 only when every count bit is 1.
3. The outer AND also requires en = 1.

254, en 1 → 0. 255, en 0 → 0. 255, en 1 → 1.

CHECK assign is continuous logic; wrap is not assigned inside the clocked block.

The wrapper declares the fixed external ports

```
module tt_um_mdkamrulislam_counter (
    input  wire [7:0] ui_in,
    output wire [7:0] uo_out,
    input  wire [7:0] uio_in,
    output wire [7:0] uio_out,
    output wire [7:0] uio_oe,
    input  wire       ena,
    input  wire       clk,
    input  wire       rst_n
);
```

1. This is the actual Tiny Tapeout top-module declaration.
2. The counter's own ports are connected inside this boundary.
3. Keep these external names, directions and widths unchanged.

CHECK Do not add a counter en port at this external boundary; use ui_in[0].

Named connections make the adapter readable

```
counter #(.WIDTH(8)) counter_inst (
    .clk(clk), .rst(~rst_n), .en(ui_in[0]),
    .count(uo_out), .wrap(uio_out[0])
);
```

1. counter_inst instantiates counter with WIDTH set to 8.
2. .port(signal) names the counter port on the left.
3. clk passes through; rst_n is inverted; ui_in[0] enables counting.

CHECK **The dot-name is the counter port; parentheses hold the wrapper expression.**

Route eight count bits and one flag outward

1. .count(uo_out) connects the whole eight-bit count bus.
2. .wrap(uio_out[0]) connects one flag bit.
3. The remaining uio output data and direction are assigned separately.

Count 5 → uo_out 00000101; wrap 0 → uio_out[0] 0.

CHECK A connected uio_out bit still needs its corresponding uio_oe bit set.

Make every remaining output intentional

```
assign uio_out[7:1] = 7'b0;
assign uio_oe = 8'b0000_0001;
```

1. uio_out[7:1] gets seven known zero data bits.
2. uio_oe enables only position 0 as an output.
3. Unused output data does not leave the other pins actively driven.

CHECK Known output data and pin direction are separate decisions.

The unused-input line does not add behaviour

```
wire _unused = &{ena, ui_in[7:1], uio_in, 1'b0};
```

1. This wrapper ignores ena and unused input bits.
2. {...} concatenates those signals; & reduces them.
3. The included 1'b0 makes _unused zero for known inputs.

CHECK Do not interpret _unused as a feature or a physical output.

A physical switch needs more than a wire

1. The minimal counter assumes enable is synchronous to its clock.
2. A real asynchronous switch needs synchronisation and debouncing.
3. Use a suitable controller clock; do not clock it from a raw bouncing button.

WORKED EXAMPLE

Real switch → synchronise and debounce. Controller → synchronous inputs.

CHECK **Synchronise and debounce real switches; use a controlled clock.**

The simulator surrounds our circuit with tb

1. test/tb.v contains the simulation top module tb.
2. tb instantiates the wrapper as user_project.
3. cocotb receives tb as dut and drives its exposed signals.

Simulation hierarchy: tb → user_project → counter_inst in RTL.

CHECK dut is a simulator handle, not another hardware module declaration.

tb connects the signals Python will drive

```
tt_um_mdkamrulislam_counter user_project (
    .ui_in(ui_in), .uo_out(uo_out),
    .uio_in(uio_in), .uio_out(uio_out),
    .uio_oe(uio_oe), .ena(ena),
    .clk(clk), .rst_n(rst_n)
);
```

1. tb declares input-driving variables and output-observation wires.
2. The named connections link those signals to the wrapper.
3. The Python test reads and writes them through dut.

CHECK This is the RTL connection excerpt, not a complete standalone tb.v file.

The dump block records simulated signal history

```
initial begin
    $dumpfile("tb.fst");
    $dumpvars(0, tb);
    #1;
end
```

$dumpfile names the recorded file tb.fst. • $dumpvars selects signals to record under tb. • initial runs once in simulation; it is not the counter's reset circuit. • #1 advances simulated time; it does not create a hardware delay.

CHECK A waveform is evidence of simulated values; assertions judge expected behaviour.

Gate-level mode adds simulation power connections

```
`ifdef GL_TEST
  wire VPWR = 1'b1;
  wire VGND = 1'b0;
`endif

// Inside the user_project instance:
`ifdef GL_TEST
  .VPWR(VPWR),
  .VGND(VGND),
`endif
```

1. GL_TEST adds extra netlist connections.
2. VPWR and VGND supply logical simulation power values.
3. The original RTL wrapper keeps its fixed Tiny Tapeout interface.

CHECK Logical model power values are not board-voltage specifications.

The Python test imports simulation tools

```
import cocotb
from cocotb.clock import Clock
from cocotb.triggers import FallingEdge, RisingEdge, Timer

@cocotb.test()
async def test_project(dut):
```

Clock generates a repeating simulated clock. • Edge triggers and Timer let the test wait for specific moments. • @cocotb.test() registers an async test with a dut handle. • async / await let this test wait while simulation continues.

CHECK **This opening is an excerpt; its indented test body follows in test.py.**

Initial inputs establish a controlled starting condition

```
dut.ena.value = 1
dut.ui_in.value = 0
dut.uio_in.value = 0
dut.rst_n.value = 0
clock = Clock(
    dut.clk, 20, unit="ns"
)
cocotb.start_soon(clock.start())
model = 0
```

1. Select the project, disable counting and assert reset.
2. start_soon runs the clock alongside the rest of the test.
3. model = 0 sets the expected value; real reset edges are still tested.

CHECK **The model is Python bookkeeping; count is hardware state.**

step describes one controlled test cycle

```
async def step(enable=0, reset=False):
    nonlocal model
```

1. enable defaults to 0; reset defaults to False.
2. nonlocal model updates the enclosing test's expected state.
3. Each call drives inputs, advances one edge and checks outputs.

step(): hold. step(enable=1): count. step(reset=True): reset.

CHECK Every step is checked; repeated calls share the same expected-state model.

Drive inputs at the falling edge

```
await FallingEdge(dut.clk)
dut.ui_in.value = enable
dut.rst_n.value = int(not reset)
```

1. Wait for FallingEdge before changing enable and reset.
2. int(not reset) converts the reset request to active-low rst_n.
3. The next rising edge is 10 ns later in this 20 ns clock.

reset = True → not reset = False → rst_n = 0.

CHECK Drive before the sampling edge; do not infer hardware timing sign-off from this wait.

Observe the edge, then allow outputs to settle

```
await RisingEdge(dut.clk)
model = 0 if reset else ((model + enable) & 0xFF)
await Timer(8, unit="ns")
```

1. RisingEdge reaches the counter’s update event.
2. The simulator still has scheduled HDL updates to process.
3. This helper samples after Timer(8 ns), before the next falling edge.

Relative timing: drive at −10 ns; rising edge at 0; sample at +8 ns.

CHECK These timings explain this test; they do not establish maximum silicon frequency.

The model predicts; it does not copy count

```
model = 0 if reset else ((model + enable) & 0xFF)
```

1. If reset is True, predict 0 regardless of enable.
2. Otherwise add the 0-or-1 enable to the previous model.
3. Mask with 0xFF to keep only eight bits.

(255 + 1) & 0xFF = 256 & 255 = 0.

CHECK Do not replace the model calculation with model = int(dut.uo_out.value).

An assertion compares observed and expected values

```
assert int(dut.uo_out.value) == model, f"count: expected {model}"
```

Read uo_out after the settling wait. • Convert the bus to an integer and compare it with model. • A mismatch fails the test instead of silently continuing. • EXAMPLE: Hypothetical: expected 2, observed 1 → investigate the first mismatching step.

CHECK The real worked-example test passed; this mismatch is a teaching scenario.

Check the flag, unused data and direction too

```
expected_wrap = int(bool(enable) and model == 255)
assert int(dut.uio_out.value) == expected_wrap, (
    "wrap or unused output"
)
assert int(dut.uio_oe.value) == 1, (
    "only uio[0] should drive an output"
)
return expected_wrap
```

Expected wrap is 1 only if enable is nonzero and the updated model is 255. • The whole uio_out bus must equal 0 or 1. • The whole uio_oe bus must equal 1: only bit 0 drives. • Return the flag only after its observed value passes the check.

CHECK The flag is checked after the edge; wrap at newly reached 255 is 1 if enabled.

Phase 1 checks reset, then hold at zero

```
for _ in range(3):
    await step(reset=True)
for _ in range(5):
    await step()
```

Three steps keep reset active and check count = 0. • Five default steps release reset and hold count = 0. • Every step also checks wrap = 0 and the direction mask. • range(3) repeats three times; _ ignores the loop index. • EXAMPLE: After this phase: known count 0, wrap 0, reset released, enable low.

CHECK A default step is an actively checked hold cycle, not an omitted test.

Phase 2 crosses overflow, holds, then resumes

```
wraps = 0
for _ in range(266):
    wraps += await step(enable=1)
assert wraps == 1, f"expected one wrap indication, saw {wraps}"
for _ in range(20):
    await step()
for _ in range(7):
    await step(enable=1)
```

266 increments from 0 end at 10 and check one high flag sample. • 20 disabled steps hold 10 while the clock runs. • 7 more enabled steps end at 17. • wraps += adds each checked flag sample; its total must be 1. • EXAMPLE: 0 + 266 modulo 256 = 10; hold 10; 10 + 7 = 17.

CHECK A single reported test can include hundreds of checked cycles.

Phase 3 resets a running nonzero counter

```
await step(enable=1, reset=True)
await step()
for _ in range(5):
    await step(enable=1)
```

1. Starting at 17, request reset and enable together.
2. The next count must be 0: reset has priority.
3. Hold once, then five enabled steps must reach 5.

Phase entry 17 → reset + enable: 0 → hold: 0 → five counts: 5.

CHECK Reset should work after counting, not only before the first increment.

Phase 4 holds at terminal count and suppresses wrap

```
await step(reset=True)
for _ in range(255):
    await step(enable=1)
for _ in range(3):
    await step()
await step(enable=1)
assert model == 0
```

1. Reset, then 255 enabled steps reach count 255 with wrap high.
2. Three disabled steps retain 255 but require wrap = 0.
3. The next enabled step overflows to 0; model must finish at 0.

CHECK Do not call wrap an after-overflow pulse; the final settled wrap is 0.

One test function checks many phases

One reported test includes reset, hold, count, overflow and flag masking.

A passing result supports the behaviours actually driven and asserted.

It does not prove every possible input timing or physical behaviour.

1 test function ≠ 1 input case. Read the loops and assertions.

CHECK Inspect the reported result for the intended commit, not just a green icon.

Final exercise: predict count, wrap and direction

TRY IT FIRST

Start at count 254, with reset released.
Take edges: enabled, disabled, re-enabled.

Predict count, wrap and uio_oe after each edge.

WORKED ANSWER

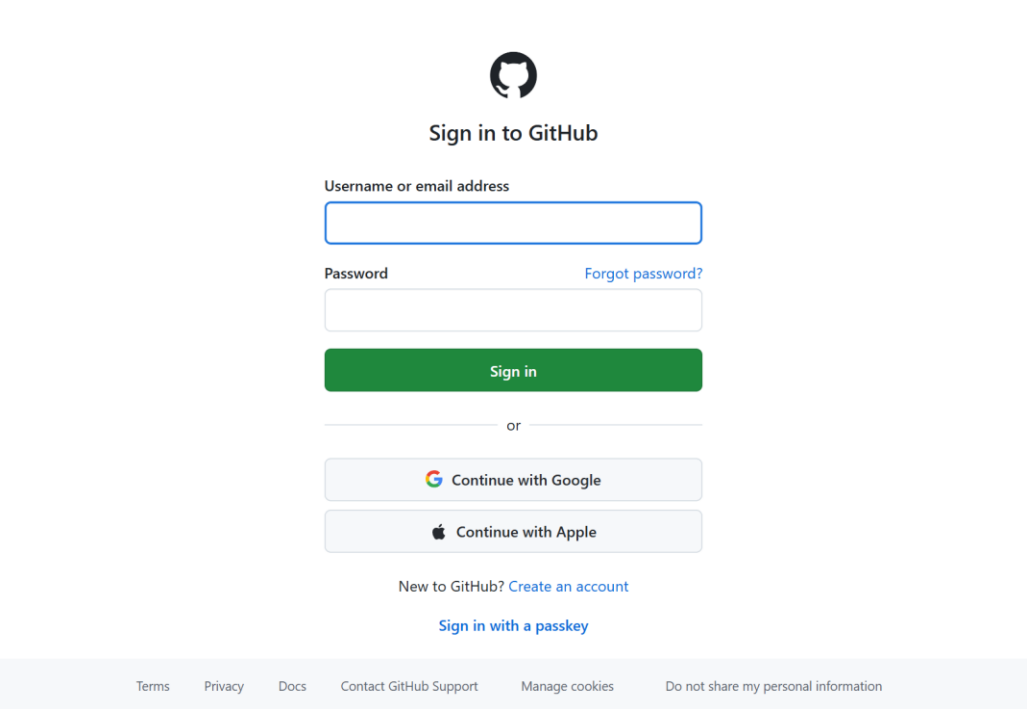
Count/wrap: 255/1, 255/0, 0/0. uio_oe stays 00000001 throughout.

Your browser lab: from files to checked layout

1. Start with a browser, a free GitHub account and the starter ZIP.
2. Copy the SKY template → add seven files → test → build layout.
3. Read the waveform, datasheet and reports; archive one revision.

CHECK Success means checked, archived design files; fabrication is a separate decision.

Step 0: sign in to your own GitHub account



Actual public GitHub sign-in screen, 8 October 2026; enter your own details.

1. Open github.com and sign in; new users choose Create an account.
2. Use a free personal account and complete email verification.
3. Check the account menu shows your username.

CHECK You can identify your signed-in account before creating a repository.

Step 1: predict the counter at a clock edge

1. rst_n=0: reset on the next rising edge.
2. Reset released, ui_in[0]=1: increment on that edge.
3. Reset released, ui_in[0]=0: hold on that edge.
4. uo_out[7:0] carries the binary count.

WORKED EXAMPLE

After reset: enabled edge → 1; disabled edge → 1; reset edge → 0.

CHECK Say when the value changes, rather than only naming its next value.

Step 1: watch the flag before overflow

Enabled count 254 → 255 → 0 on successive rising edges.

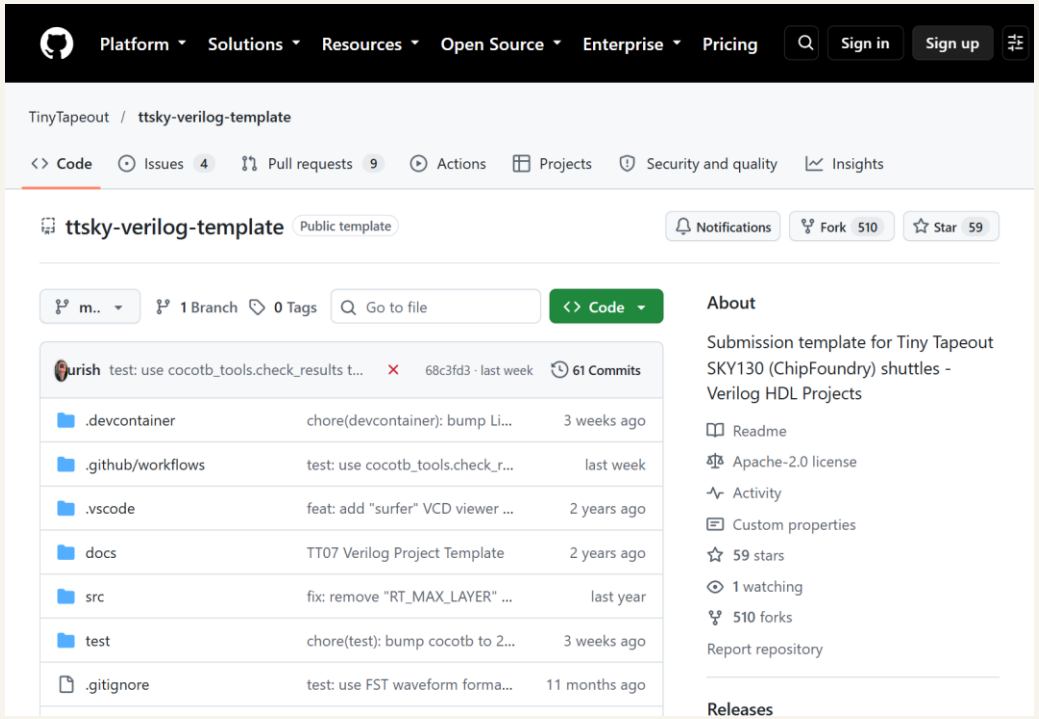
wrap = 1 while count is 255 and enable is 1.

Disable at 255: count holds at 255; wrap becomes 0.

count=255, enable=0 → uo_out=11111111, wrap=0.

CHECK Predict both count and wrap; they follow different update rules.

Step 2: open the official SKY template

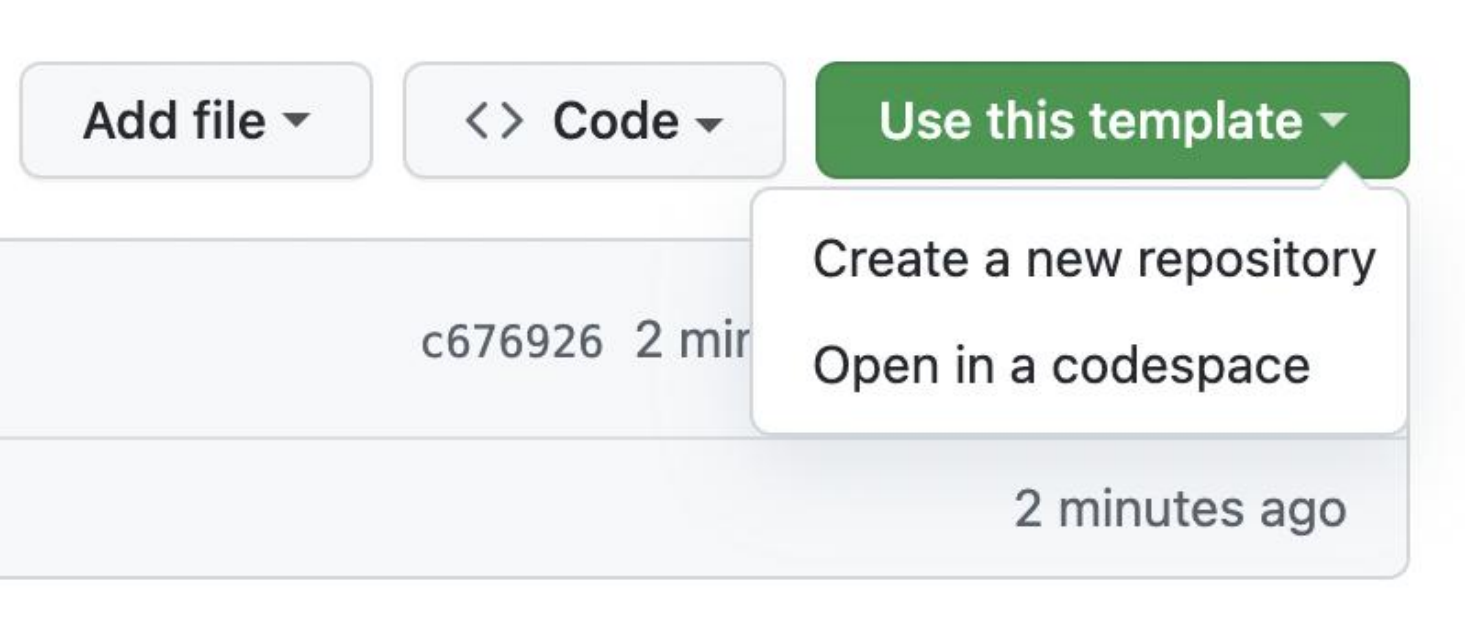


Actual official SKY template page, captured 8 October 2026.

1. Open tinytapeout.com/hdl/templates/.
2. Choose the SKY Verilog template.
3. Confirm TinyTapeout / ttsky-verilog-template in the header.

CHECK You have opened the SKY Verilog template, before making your own copy.

Step 3: create a repository from the template

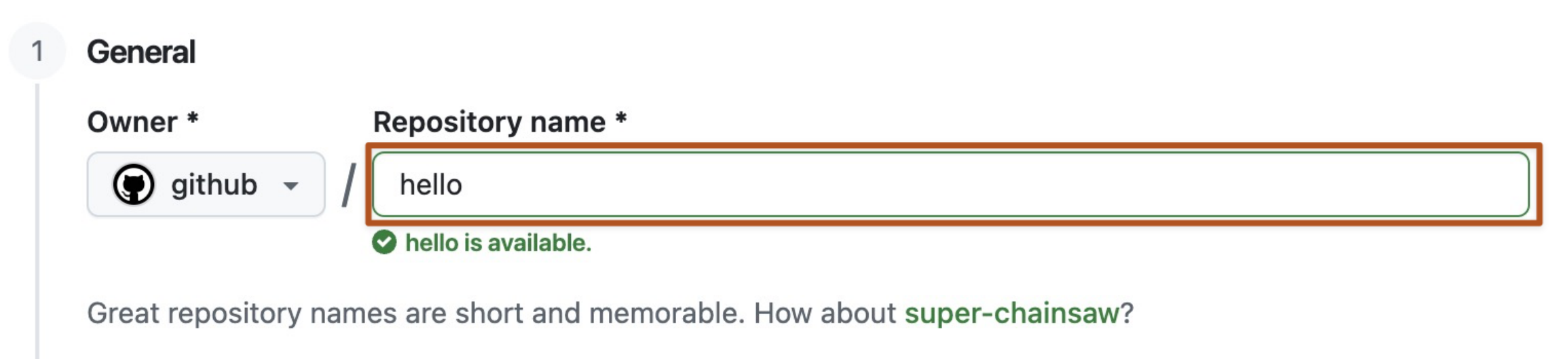


1. While signed in, select Use this template.
2. Choose Create a new repository.
3. Continue to the owner, name and visibility form.

Official GitHub Docs image: template-copy menu; instructional account-only example.

CHECK The creation form will make a copy under your account.

Step 3: choose your owner, name and Public

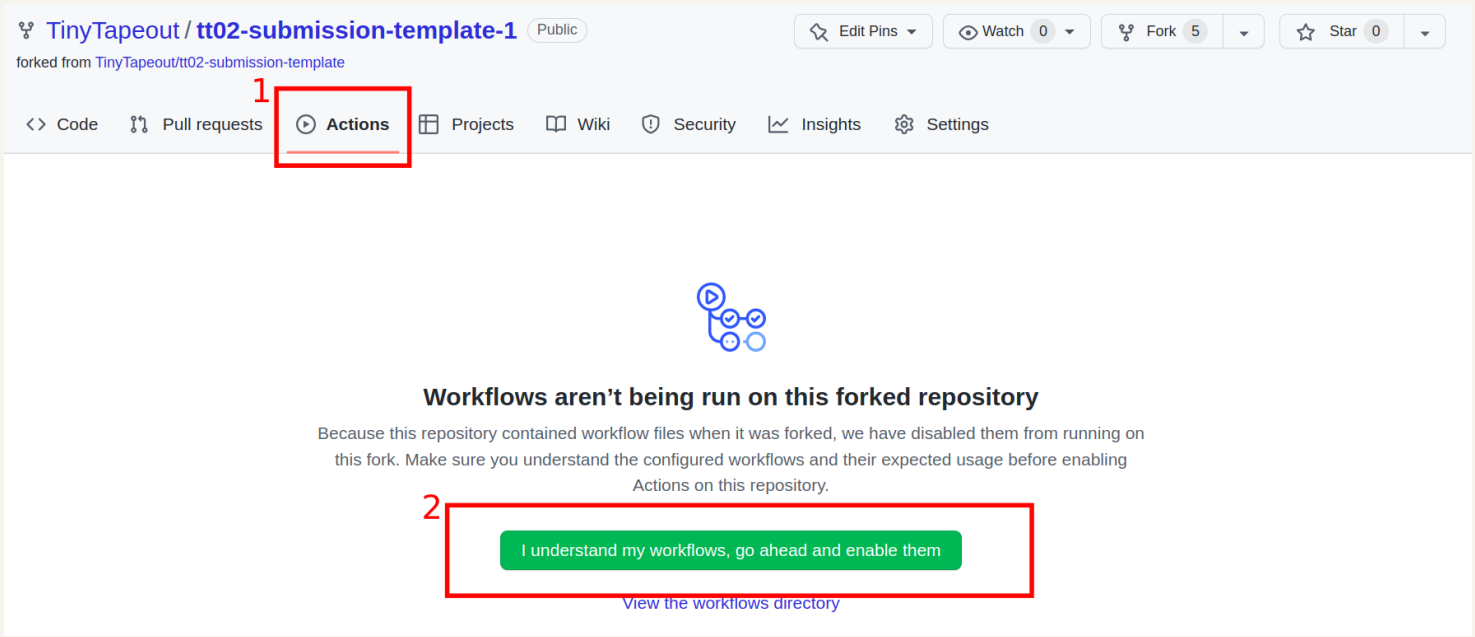


Owner: your account. Name: for example tt-counter-lesson. • Select Public; the default branch alone is sufficient. • Create the repository; confirm your username in its header.

Official GitHub Docs image: repository-name field; use your own account and name.

CHECK The URL is github.com/YOUR-USERNAME/YOUR-REPOSITORY.

Step 4: make sure Actions is enabled

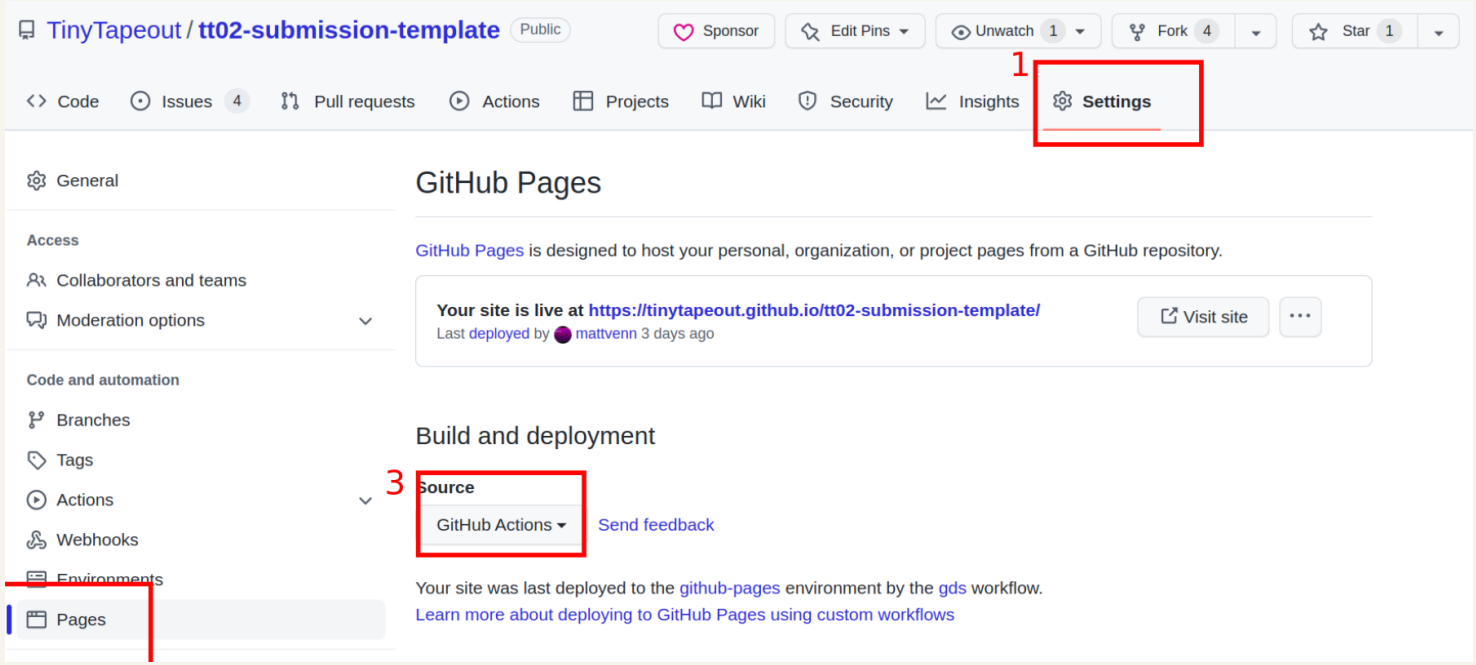


Official Tiny Tapeout FAQ image: Actions enablement in an older example repository.

1. Open the Actions tab in your new repository.
2. If an enablement banner appears, enable the workflows.
3. If workflows are already available, continue to Pages.

CHECK The repository can run its template workflows.

Step 4: set Pages Source to GitHub Actions



1. Open Settings → Pages.
2. Under Build and deployment, set Source to GitHub Actions.
3. Use the template publishing workflow already provided.

Official Tiny Tapeout FAQ image: Pages Source = GitHub Actions; older repository shown.

CHECK Source displays GitHub Actions before the complete GDS build.

Step 5: extract the starter, then open its contents

1. Download counter-starter.zip from the lesson files.
2. Extract it on your computer; open the extracted folder.
3. Find info.yaml beside src, test and docs.
4. Show filename extensions: counter.v must not be counter.v.txt.

CHECK You see the root YAML file and three folders, rather than a nested starter directory.

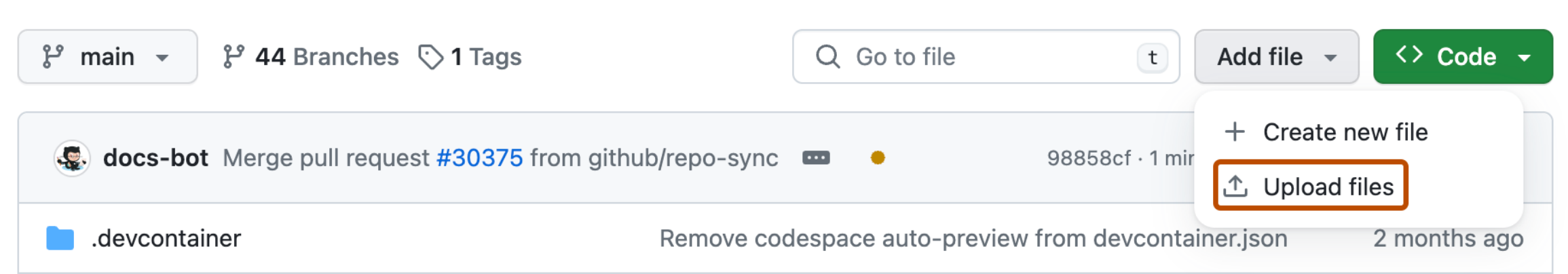
Step 5: count the seven files

```
info.yaml
src/counter.v
src/project.v
test/Makefile
test/tb.v
test/test.py
docs/info.md
```

1. src: counter logic and the Tiny Tapeout wrapper.
2. test: source list, Verilog testbench and Python assertions.
3. docs: datasheet text; info.yaml: metadata and pin labels.

CHECK Two + three + one + one = exactly seven files.

Step 6: open the browser upload page



In your own repository, open Code and the root file list. • Choose Add file → Upload files. • Prepare to upload the extracted contents with their folders.

Official GitHub Docs image: Add file control used to open Upload files.

CHECK The upload page belongs to your copy and its root directory.

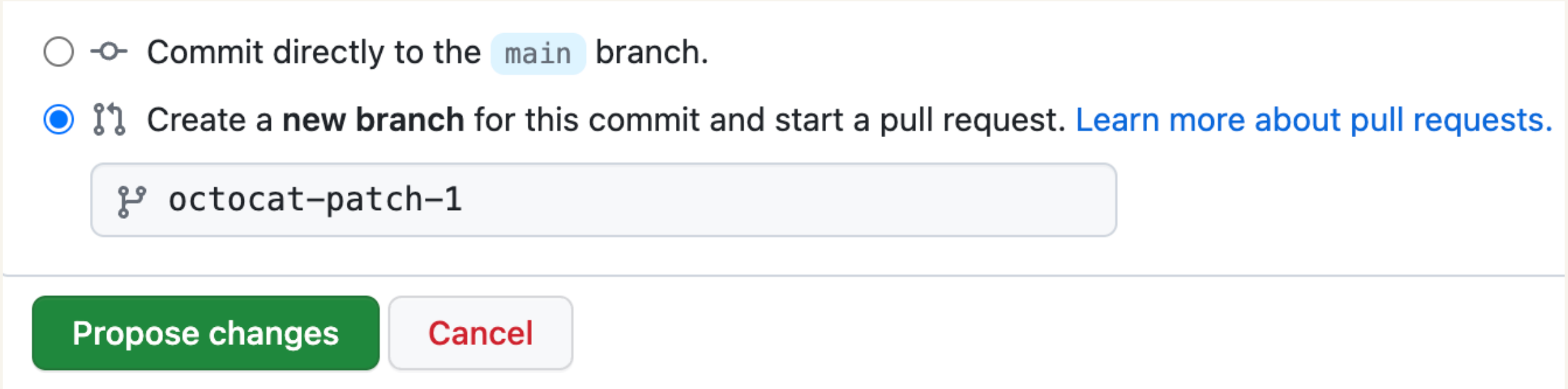
Step 6: drag the contents, preserving folders

```
info.yaml
src/counter.v
src/project.v
test/Makefile
test/tb.v
test/test.py
docs/info.md
```

1. Drag info.yaml plus the src, test and docs folders.
2. Inspect every displayed upload path before saving.
3. Remove an unwanted outer starter-folder prefix.

CHECK All seven listed paths match before you commit.

Step 6: save one complete counter commit



Message: Add eight-bit counter lesson. • For your personal lab, choose Commit directly to main. • Save the changes; follow a required PR route if branch rules apply.

Official GitHub Docs branch-choice example; new branch shown selected. Use your lab's required branch route.

CHECK The Code file list shows the new commit containing all seven files.

Alternative: replace an existing file in the editor

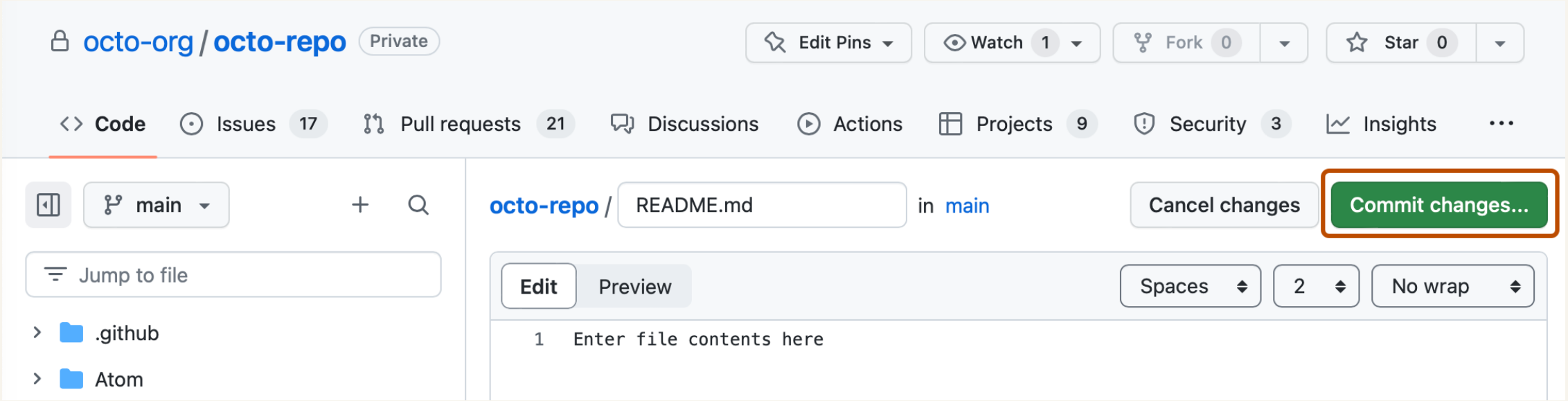


Open a file and select its pencil / Edit file control. • Paste that file's complete Appendix A source. • Commit changes... with a message to the intended branch.

Official GitHub Docs image: edit-file pencil; instructional account-only example.

CHECK Saved contents match the complete appendix file, not a slide excerpt.

Alternative: add the new counter module

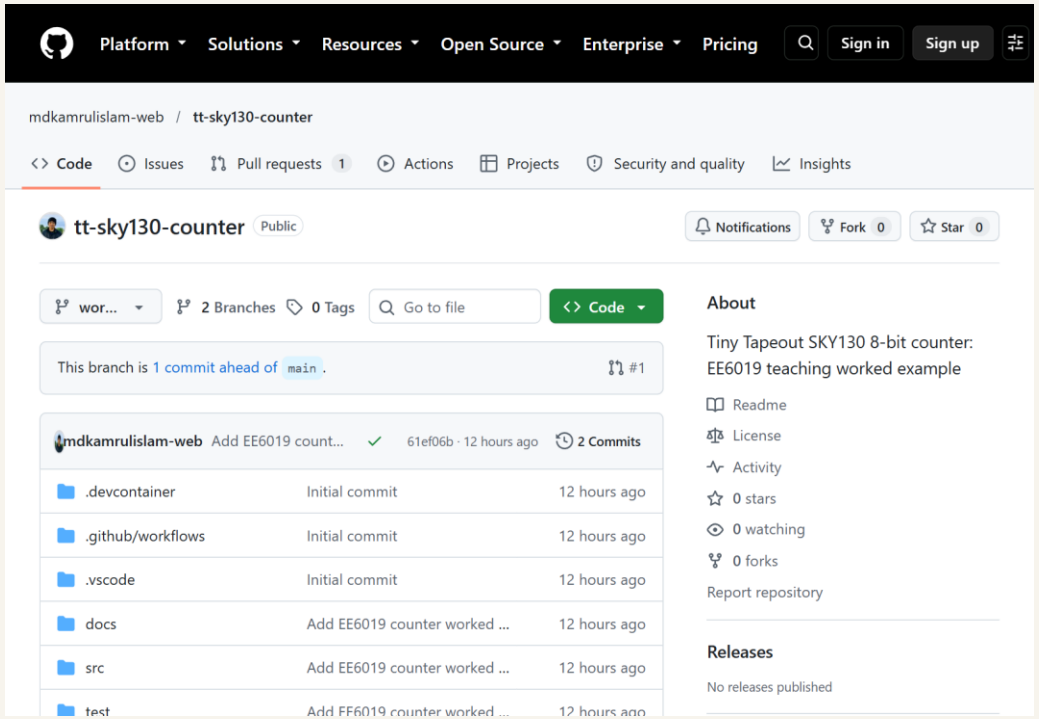


Open src → Add file → Create new file. • Name it counter.v; paste the complete Appendix A.2 source. • Select Commit changes… and save; complete the other six files too.

Official GitHub Docs image: Commit changes button opens the form; example is not our counter account.

CHECK src contains both counter.v and project.v.

Step 6 checkpoint: inspect your final tree and version



Actual reference tree on worked-example at 61ef06b; your copy has its own owner, branch and commit.

1. Open Code: verify src, test, docs and root info.yaml.
2. Confirm counter.v and project.v are both inside src.
3. Record the final branch and commit identifier.

CHECK One final saved version contains all seven example files.

Step 7: read title, author and description

```
title:       "8-bit counter (EE6019 worked example)"      # Project title
author:      "MD. KAMRUL ISLAM"      # Your name
discord:     ""      # Your discord username, for communication and automat
description: "8-bit synchronous counter with enable, reset and wrap"      #
language:    "Verilog" # other examples include SystemVerilog, Amaranth, VH
```

These fields identify the circuit and credit its author. • Describe the implemented behaviour; keep the supplied YAML indentation.

Native detail from the actual counter info.yaml at 61ef06b: title, author and description.

CHECK Metadata is meaningful and agrees with the reference circuit and clock target.

Step 7: confirm the clock target and tile size

```
clock_hz:     50000000        # Clock frequency in Hz (or 0 if not applicable

# How many tiles your design occupies? A single tile is about 167x108 uM.
tiles: "1x1"            # Valid values: 1x1, 1x2, 2x2, 3x2, 4x2, 6x2 or 8x2
```

clock_hz = 50000000 means 50 MHz, or a 20 ns cycle. • tiles = 1x1 sets one layout tile for this example. • Keep the clock metadata, test clock and timing constraints consistent.

Native detail from the actual counter info.yaml at 61ef06b: clock_hz and tiles.

CHECK Metadata is meaningful and agrees with the reference circuit and clock target.

Step 7: check both source-file lists

```
# info.yaml excerpt
project:
  source_files:
    - "project.v"
    - "counter.v"

# test/Makefile excerpt
PROJECT_SOURCES = project.v counter.v
```

1. In info.yaml, list both Verilog files under project.source_files.
2. In test/Makefile, include both in PROJECT_SOURCES.
3. These entries are filenames for files already inside src.

CHECK Both lists name project.v and counter.v, and both files exist.

Step 7: match the top name in three places

info.yaml: project.top_module.

src/project.v: the wrapper's module declaration.

test/tb.v: the module instantiated as user_project.

All three use tt_um_mdkamrulislam_counter; the inner module is counter.

CHECK Compare the exact spelling in all three locations.

Step 7: confirm the wrapper's pin mapping

```
counter #(.WIDTH(8)) counter_inst (
    .clk(clk), .rst(~rst_n), .en(ui_in[0]),
    .count(uo_out), .wrap(uio_out[0])
);
assign uio_out[7:1] = 7'b0;
assign uio_oe = 8'b0000_0001;
```

Keep ui_in, uo_out, uio_in, uio_out, uio_oe, ena, clk, rst_n. • ui_in[0] enables counting; ena selects the project. • uo_out carries count; uio_out[0] carries wrap; uio_oe = 01.

Actual counter wrapper at 61ef06b; inspect connections and the output-enable mask.

CHECK Enable, count, wrap and pin directions agree with the specification.

Step 7: write a datasheet someone can use

1. Open docs/info.md.
2. How it works: synchronous active-low reset, count, hold and wrap.
3. How to test: drive pins and predict outputs.
4. External hardware: simulation needs none; later switches need care.

WORKED EXAMPLE

“After a reset edge, ui_in[0]=1 increments; ui_in[0]=0 holds.”

CHECK A reader can drive the named pins and predict an observable value.

Step 8: read how the test drives and samples

```
await FallingEdge(dut.clk)
dut.ui_in.value = enable
dut.rst_n.value = int(not reset)
await RisingEdge(dut.clk)
model = 0 if reset else ((model + enable) & 0xFF)
await Timer(8, unit="ns")  # allow the gate-level out
```

Open test/test.py; find the step() helper. • Inputs change at a falling edge; state updates at the rising edge. • The checker waits 8 ns, then compares outputs with its model.

Actual reference Python test at 61ef06b; read drive/edge/settle/check order.

CHECK You can explain why driving and sampling occur at different times.

Step 8: know what the assertions cover

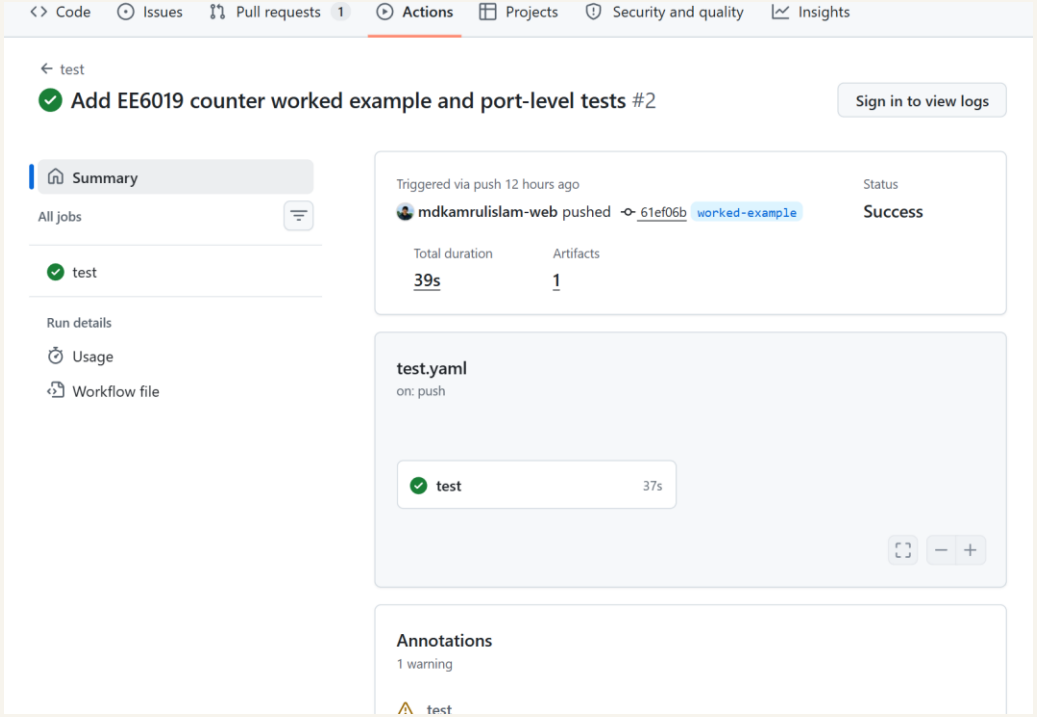
1. Check reset, enabled counting, hold and reset priority.
2. After 266 enabled edges from 0, count is 10.
3. At 255, disable → hold/flag 0; enable → overflow to 0.
4. Also check unused output bits and uio_oe.

WORKED EXAMPLE

20 disabled edges at count 10 must leave count at 10.

CHECK Explain a checked boundary case, not merely “the test passed”.

Step 9: open the test run for your final commit

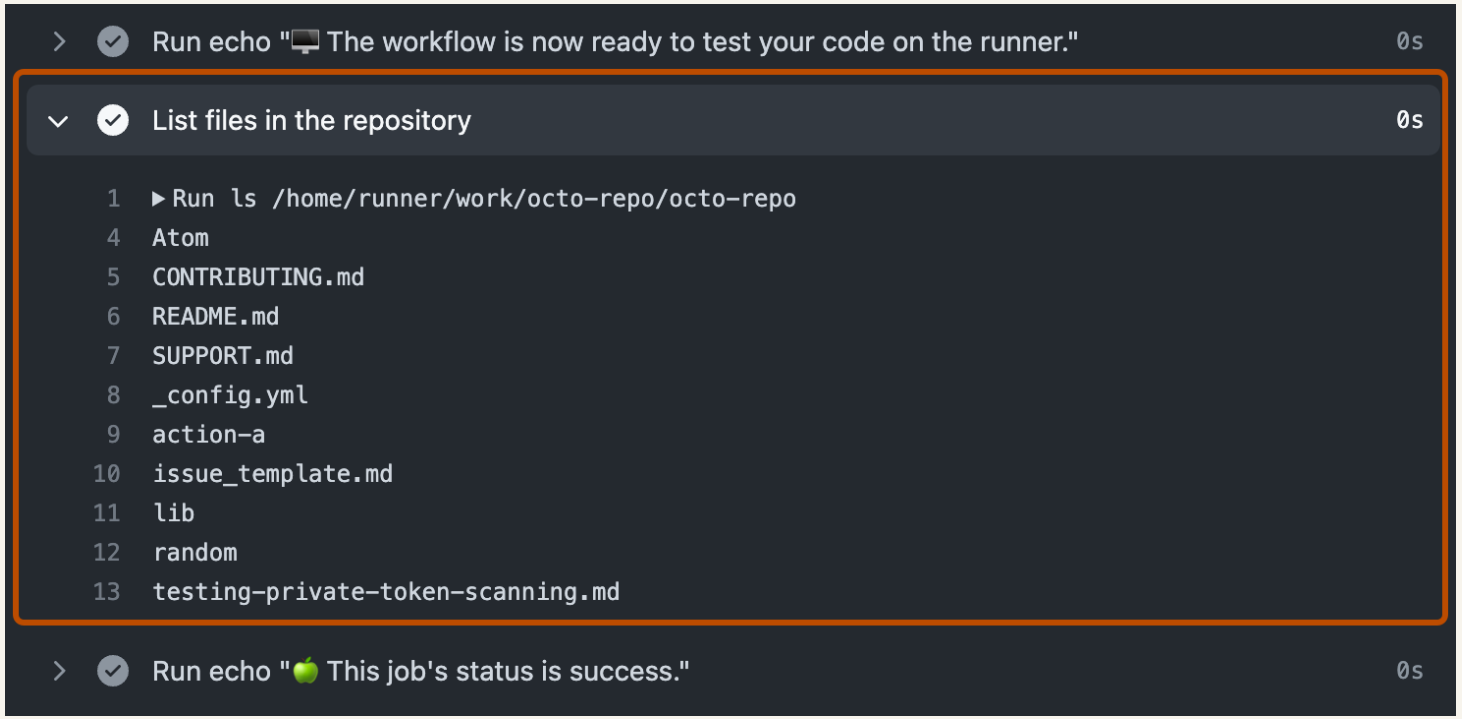


1. Actions → test → run for the complete seven-file commit.
2. Check the branch and commit identifier.
3. Open the test job; expand Run tests.

Actual successful RTL run 37693100752 at 61ef06b; open its test job to inspect logs.

CHECK The complete counter commit has a completed successful test run.

Step 9: read the job log, not only the icon



1. Select the test job in the run's sidebar.
2. Expand Run tests and read the pass/fail summary.
3. If red, find the first useful compiler or assertion error.

Official GitHub Docs image: expanded job log; generic workflow, not a counter failure.

CHECK You can locate the test summary or the first specific failure message.

Step 9: keep and read results.xml

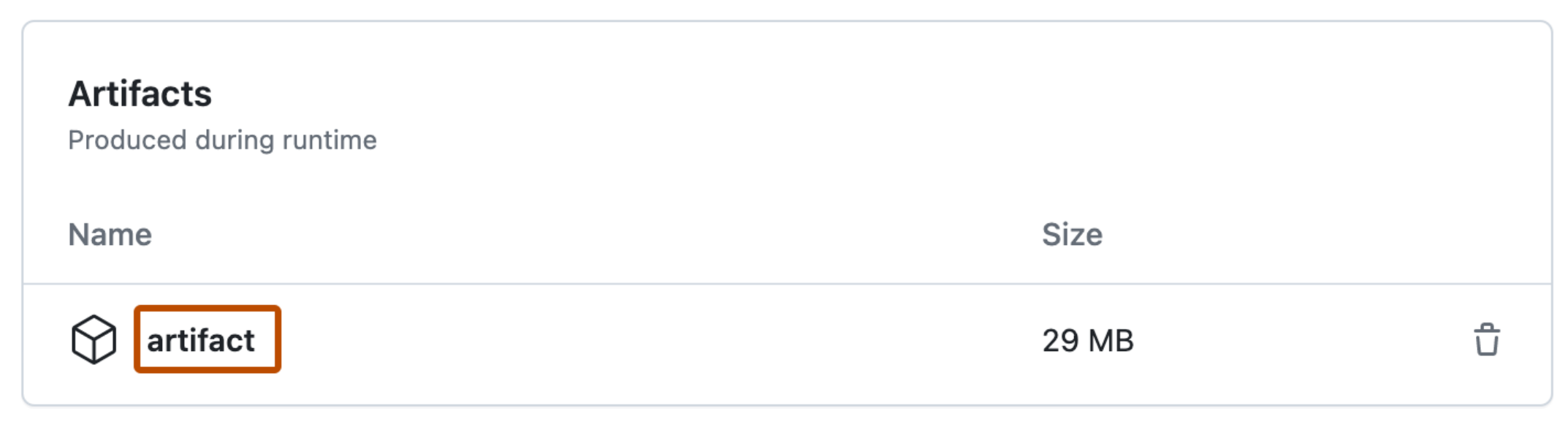
Field	Value from results.xml
name	test
tests	1
failures	0
errors	0

From Summary → Artifacts, download test-results and extract it. • Read results.xml: reference = 1 test, 0 failures, 0 errors. • Keep XML with tb.fst, the commit and run URL.

Local HTML view of unchanged results.xml from RTL run 37693100752, 61ef06b; not GitHub UI.

CHECK The XML belongs to the same successful RTL run as your saved source.

Waveform: download and extract test-results



Open the successful test run → Summary → Artifacts. • Click test-results while signed in; extract the ZIP. • Find tb.fst and results.xml; keep the originals.

Official GitHub Docs artifact-list image; choose test-results in your actual RTL run.

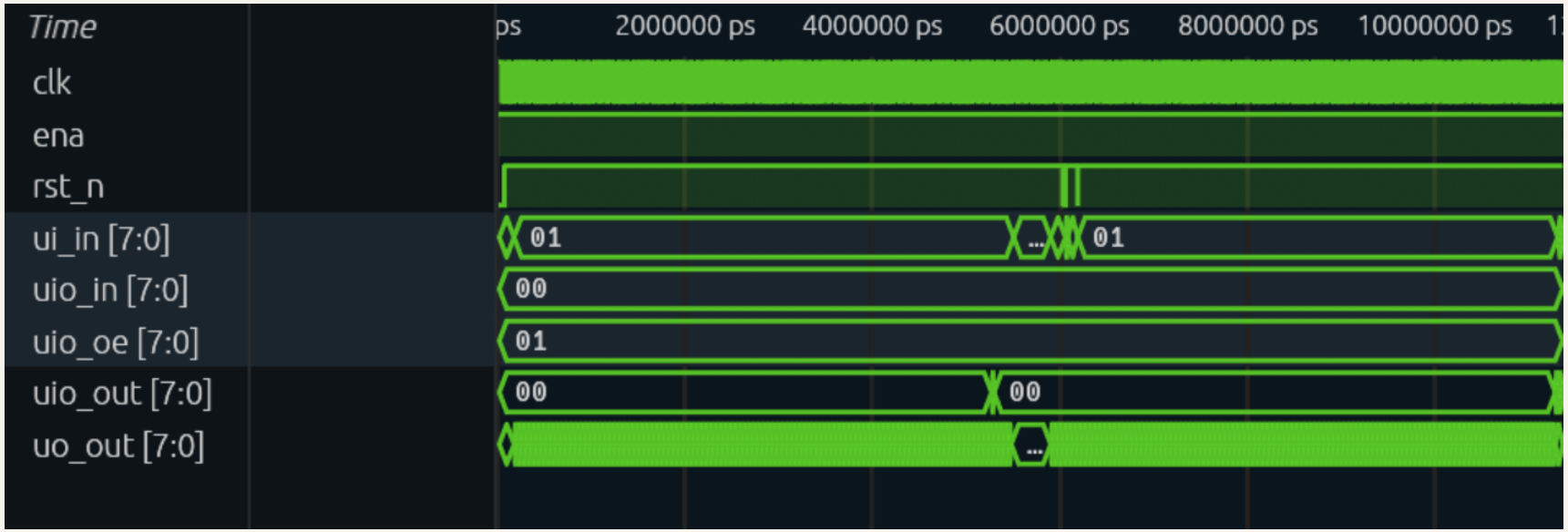
CHECK You have extracted tb.fst, not just downloaded an unopened ZIP.

Waveform: open Surfer and drop the FST

1. Open app.surfer-project.org in your browser.
2. Drag the extracted tb.fst onto the viewer canvas.
3. Wait for loading; look for tb scope and the loaded filename.

CHECK The filename is loaded and tb appears; next add signals to the wave area.

Waveform: add tb signals and fit the whole test

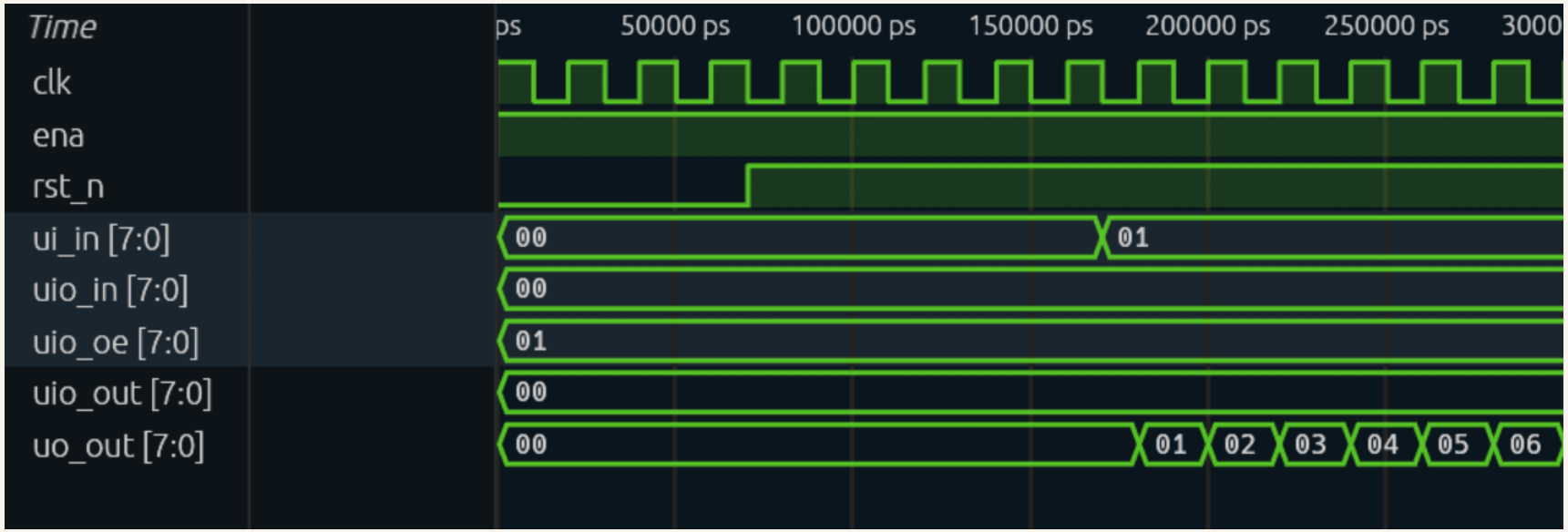


Space → type scope_add tb → Enter. • Space → type zoom_fit → Enter. • Find clk, rst_n, ui_in, uo_out, uio_out and uio_oe.

Native Surfer capture of genuine RTL tb.fst, run 37693100752, 61ef06b; full 11,368 ns test.

CHECK The wave area contains signals; a dense full view is ready to zoom.

Waveform: inspect reset and first increments

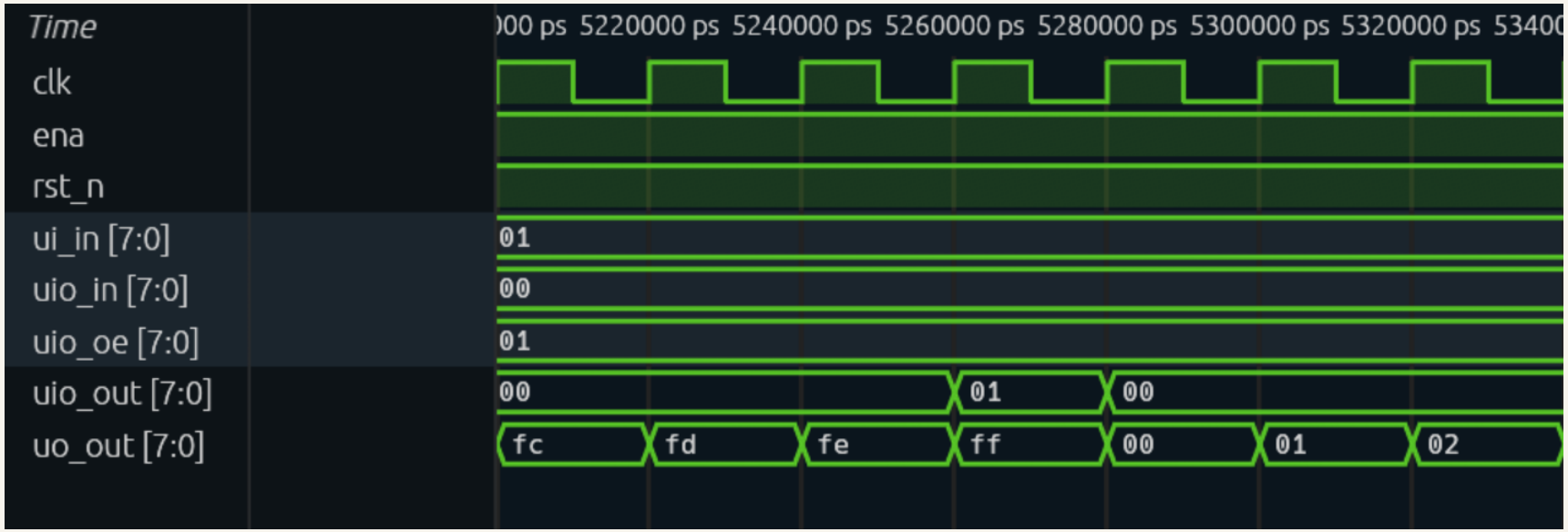


Space → enter zoom_to 0ns 300ns → Enter. • rst_n = 0: count becomes 00 at rising clock edges. • Enable 0 holds at 00; enable 1 starts increments.

Actual reset/start trace from RTL run 37693100752 at 61ef06b; buses shown in hexadecimal.

CHECK Reset clears on an edge; enabled counting begins only after reset is released.

Waveform: inspect 254 → 255 → 0

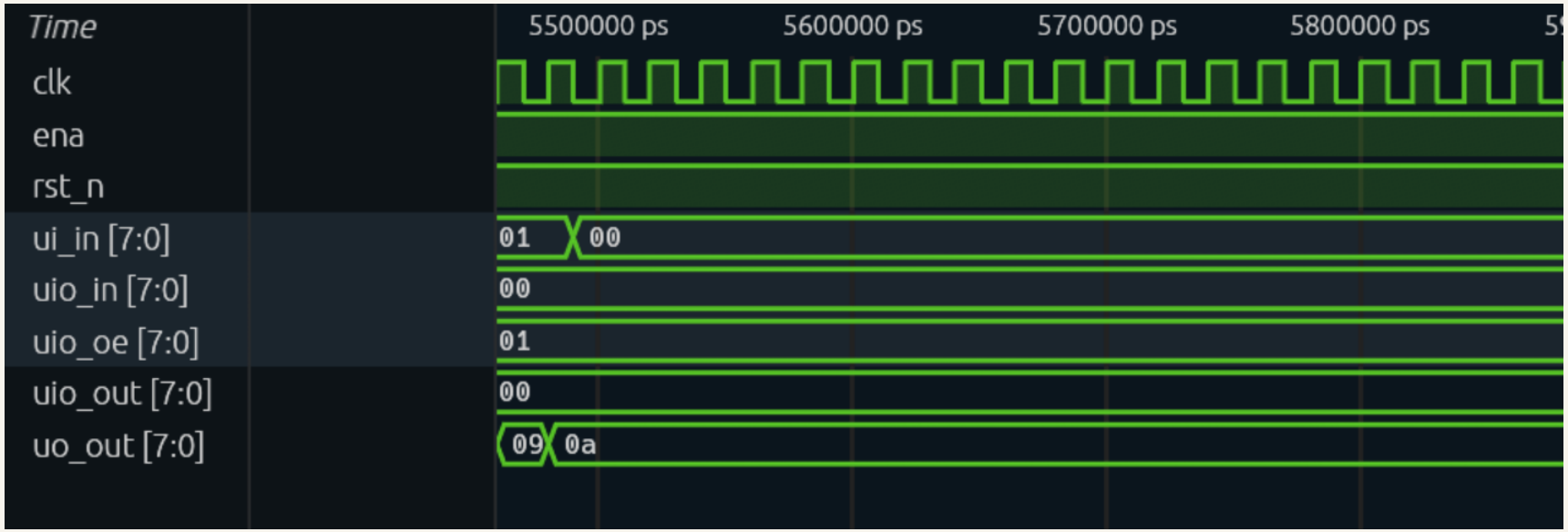


Space → enter zoom_to 5200ns 5340ns → Enter. • uo_out: fe → ff → 00 means 254 → 255 → 0. • uio_out is 01 at enabled ff, then 00 after overflow.

Actual overflow trace from RTL run 37693100752, 61ef06b: fe/ff/00 with wrap high at ff.

CHECK Explain why the flag is high before the overflow edge.

Waveform: inspect hold while the clock runs



Space → enter zoom_to 5460ns 5880ns → Enter. • ui_in changes from 01 to 00: counting becomes disabled. • uo_out stays at 0a, decimal 10, while clk keeps toggling.

Actual hold trace from RTL run 37693100752 at 61ef06b; 0a means decimal 10.

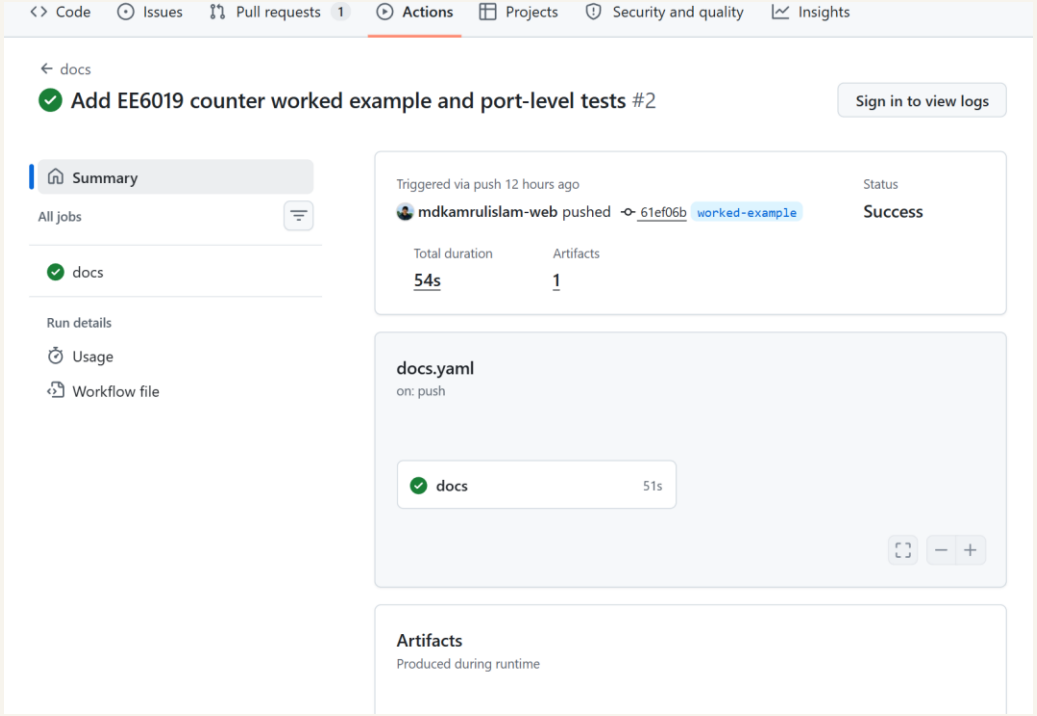
CHECK Clock edges continue, but the disabled register holds 10.

Waveform: recover an empty or confusing view

1. Check tb.fst loaded; add tb signals, then use zoom_fit.
2. Hex: 0a = 10, fe = 254, ff = 255; change radix if useful.
3. Use explicit ns in zoom_to: 1 ns = 1,000 ps.
4. For modified tests, find events rather than copy reference times.

CHECK You can see the loaded signals and recognise one reset, count or hold event.

Step 10: build and download the datasheet



1. Actions → docs → run for your final commit.
2. Wait for success; Summary → Artifacts → PDF.
3. Extract the download and open datasheet.pdf.

Actual successful docs run 37693100755 at 61ef06b; its PDF artifact contains a two-page datasheet.

CHECK Your final commit has a generated datasheet you can open.

Step 10: read reset and enable in the datasheet

How it works

An 8-bit synchronous counter increments on each rising edge of `clk` while `ui[0]` (`en`) is high. `uo[7:0]` shows its value, from 0 to 255, then back to 0. When enable is low the count holds. Driving `rst_n` low resets the count to zero on the next rising edge; reset takes priority over enable.

Read how reset and enable affect count at a rising edge. • Compare these words with the RTL and the top-level pin names.

Native crop of the actual generated datasheet, page 1; reference docs run 37693100755.

CHECK The generated document describes the same circuit that passed the test.

Step 10: check the flag description against the code

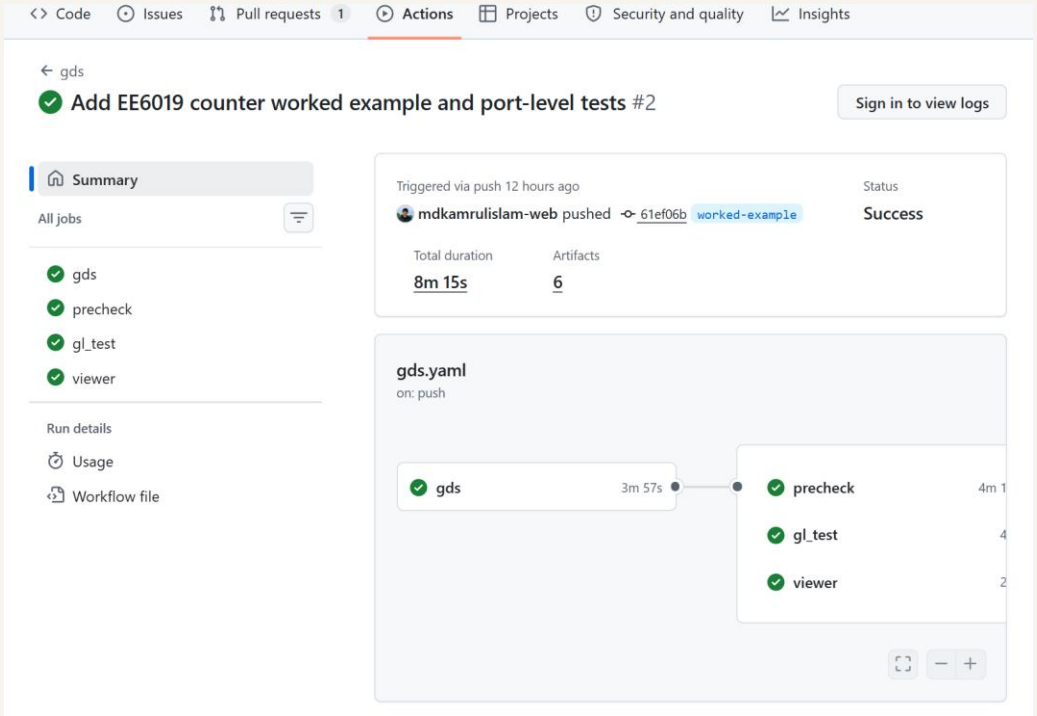
`uio[0]` (`wrap`) is high while count is 255 and enable is high. During continuous counting this lasts one clock period every 256 clocks, before the overflow edge. It is combinational, rather than a registered pulse after overflow. `uio[7:1]` are inputs and their output data is tied to zero.

The flag is high at count 255 while enable is high, before the overflow edge. • Its combinational behaviour must agree with both the RTL and your test.

Native crop of the actual generated datasheet, page 1; reference docs run 37693100755.

CHECK Documentation explains the circuit; passing simulation verifies the tested behaviour.

Step 11: wait for all four GDS jobs



1. Actions → gds → run for your final commit.
2. Wait for gds, gl_test, precheck and viewer to finish.
3. All four must pass; queued or running is still in progress.

Actual GDS run 37693100748 at 61ef06b: gds, gl_test, precheck, viewer all successful.

CHECK All four jobs passed for the same complete counter version.

Step 11: four jobs, four different questions

gds: did the tools produce the placed and wired layout?

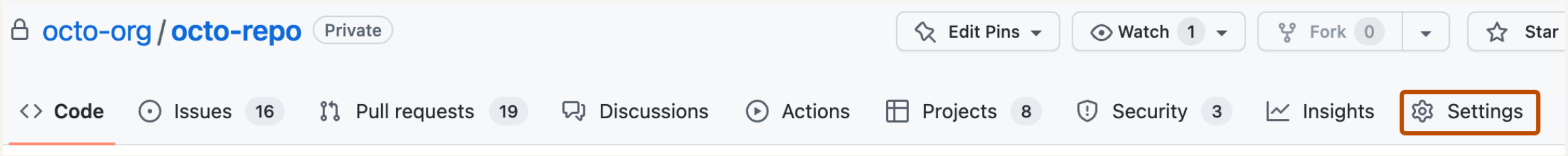
gl_test: did the built cell connections pass the functional test?

precheck: did Tiny Tapeout's required checks pass?

viewer: were the layout-viewer files published?

CHECK Name the job that checks behaviour and the job that publishes the viewer.

Step 12: open your published layout site

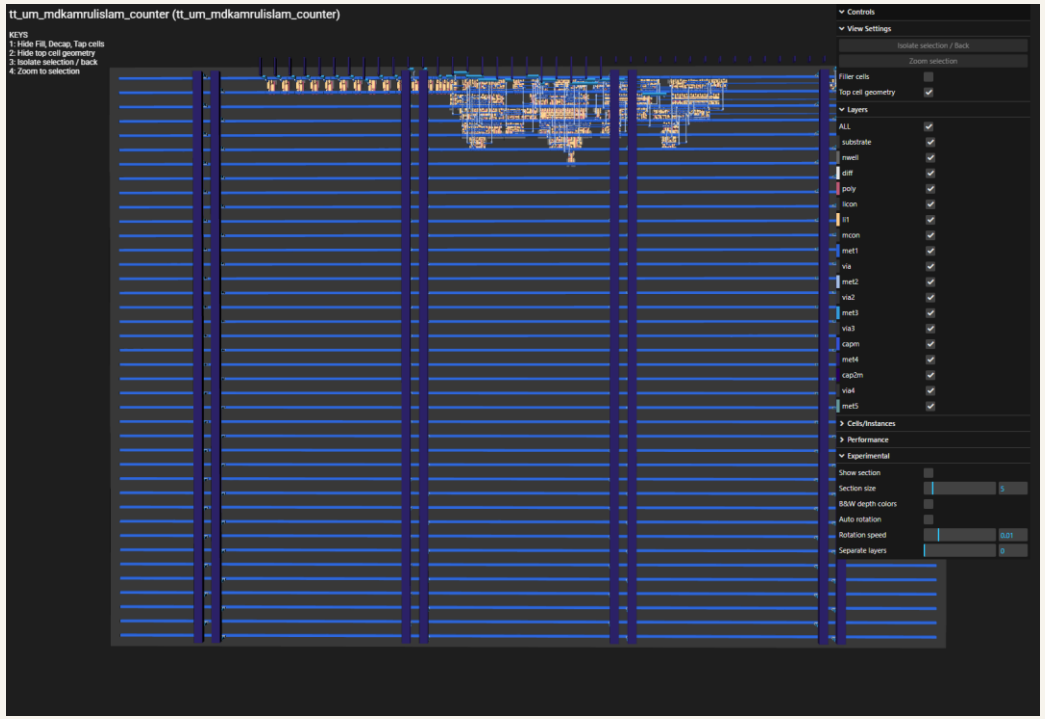


After viewer passes: Settings → Pages → Visit site. • Or open the published project URL shown in Pages. • Allow loading; confirm your top-module name and a visible layout.

Official GitHub Docs image: repository Settings tab; then choose Pages and Visit site in your copy.

CHECK A loaded layout appears under your project's top-module name.

Step 12: explore layers and hide filler cells

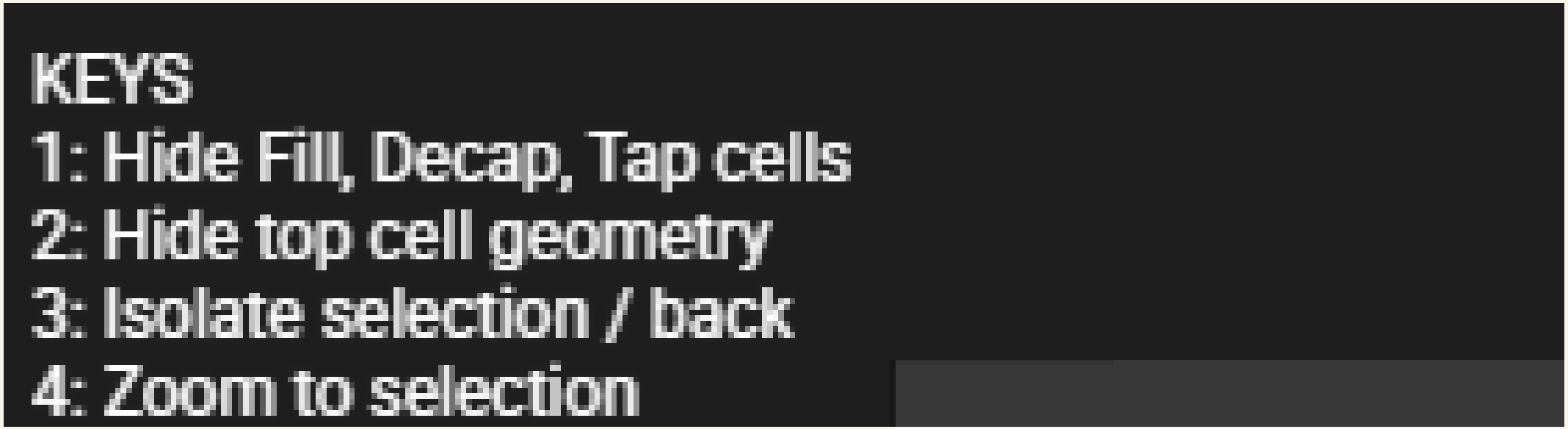


Actual native layout-viewer capture from 61ef06b; filler, decap, tap hidden using its own control.

1. Use the viewer's KEYS panel for pan, rotate and zoom.
2. Controls → View Settings → Filler cells: hide them to see logic.
3. Explore layer controls; compare the view with the build reports.

CHECK You can distinguish visible geometry from claims made by tests and reports.

Step 12: read the viewer’s keyboard controls

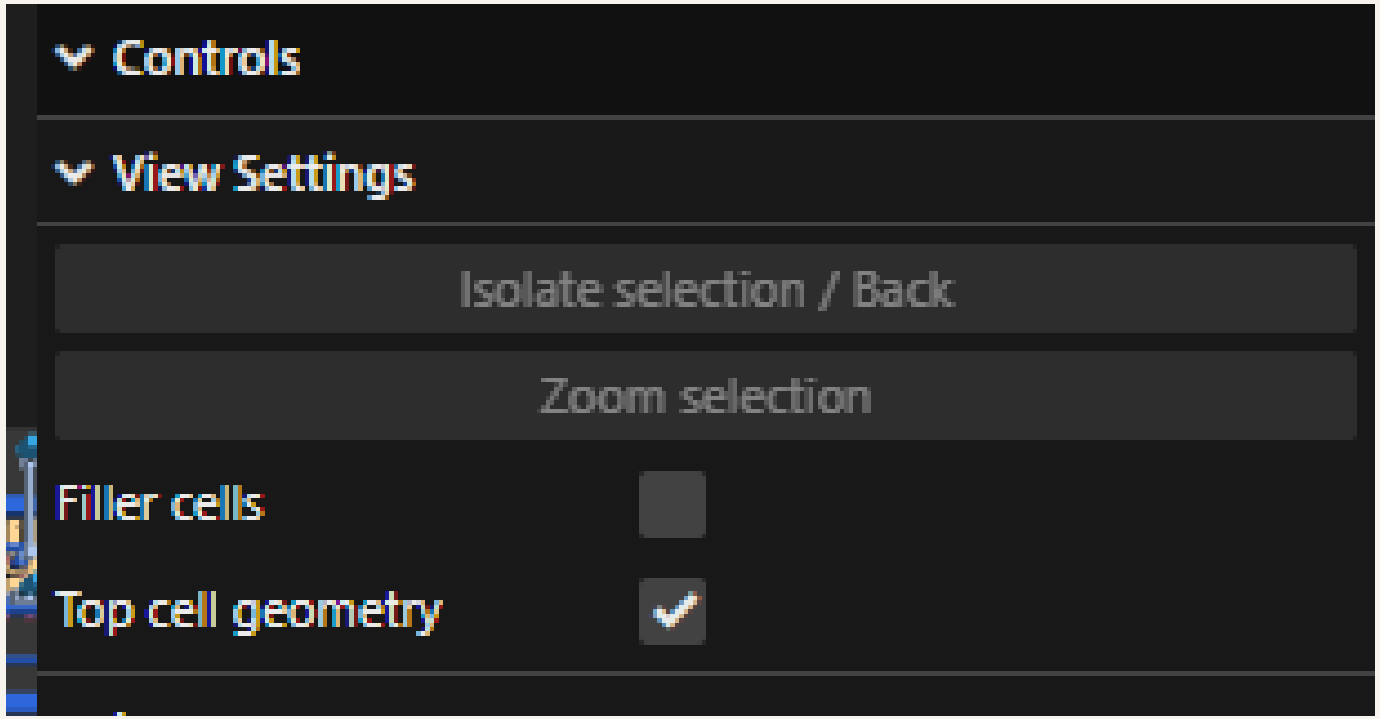


Read the actual KEYS panel: 1 hides fill, decap and tap cells. • The other keys hide top geometry, isolate a selection or zoom to it.

Native crop of the actual viewer’s KEYS panel; display choices do not change the layout file.

CHECK You can distinguish visible geometry from claims made by tests and reports.

Step 12: find the Filler cells checkbox

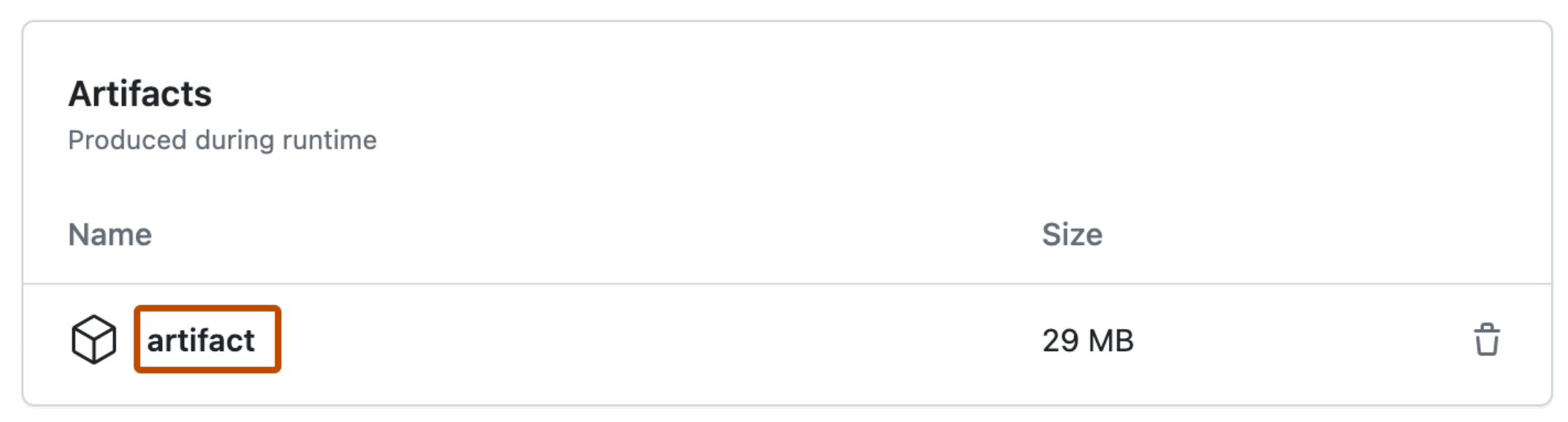


Native crop of the actual viewer's View Settings; the unchecked Filler cells control is visible.

1. Expand Controls, then View Settings.
2. Uncheck Filler cells to inspect the logic with the grouped support cells hidden.
3. The checked Top cell geometry control keeps the top-level shape visible.

CHECK You can distinguish visible geometry from claims made by tests and reports.

Step 13: download layout, image and build logs



Completed gds run → Summary → Artifacts. • gds_render: the generated 2D layout PNG. • tt_submission: layout, netlist, identity and statistics. • GDS_logs: detailed build logs and reports.

Official GitHub Docs download-control example; the actual GDS run has the artifact names listed here.

CHECK Downloaded ZIPs are extracted and labelled with their source run.

Step 13: keep the verification artifacts too

1. gatelevel_test_results: mapped-design XML and waveform.
2. precheck_reports: Tiny Tapeout check results.
3. github-pages: packaged viewer files.
4. Also keep test → test-results and docs → PDF.

CHECK RTL, gate-level, precheck, documentation and viewer files remain distinguishable.

Step 13: inspect the submission bundle's identity

```
tt_submission/
  tt_um_mdkamrulislam_counter.gds
  tt_um_mdkamrulislam_counter.oas
  tt_um_mdkamrulislam_counter.v
  commit_id.json
  pdk.json
  stats/metrics.csv
  stats/synthesis-stats.txt
```

1. Extract tt_submission; find its contained tt_submission folder.
2. Compare commit_id.json with the source/run commit.
3. Read pdk.json; keep layout, mapped Verilog and statistics together.

CHECK The bundle's identity matches the revision you tested and built.

Step 14: read counts with their stage and category

design__instance__utilization	0.0496131
design__instance__count__stdcell	279
design__instance__count	4600
design__instance__count__class:fill_cell	4321

47 cells after synthesis. • 279 final non-fill standard-cell instances, including 225 taps. • 4,600 total instances, including 4,321 filler cells. • Core utilisation: 0.0496131 × 100 ≈ 4.96%.

Local HTML view of unchanged metrics.csv from GDS run 37693100748, 61ef06b; not GitHub UI.

CHECK Every quoted count includes its build stage and what was counted.

Step 14: read timing, geometry and warnings

50 MHz target: setup/hold slack +8.64285/+0.109636 ns.

Reported setup/hold violations: 0/0; Magic DRC/LVS errors: 0/0.

Precheck: all 15 passed; review warnings separately.

CHECK Quote units and checked conditions; distinguish warnings from failures.

Step 15: start with the first useful error

1. Open the failed job and expand its first failing step.
2. Our first blank-template failure was empty metadata.
3. Finish the seven files; inspect the complete final commit.
4. Fix the cause before reading downstream skipped jobs.

CHECK Explain the error and the revision it belongs to before changing a file.

Step 15: match the symptom to its cause

Unknown module: check both source lists and all three top names.

Docs validation: check metadata, pin labels and required sections.

Count mismatch: predict reset/enable behaviour at the sampled edge.

Commit the fix; keep the assertion and inspect the new run.

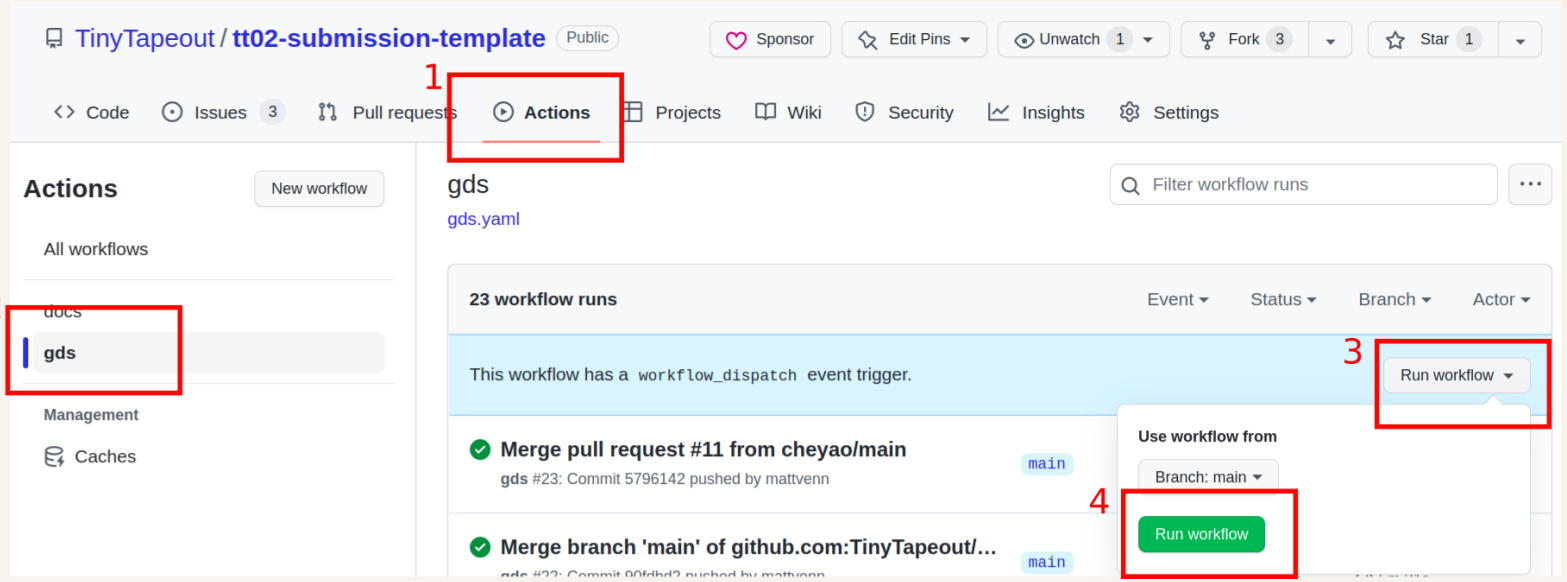
CHECK One justified fix creates one new version whose results can be evaluated.

Step 15: fix Pages or a blank viewer

1. Deployment failure: confirm Pages Source = GitHub Actions.
2. Multiple github-pages artifacts after a viewer rerun: start a fresh full GDS run.
3. Blank WebGL view: wait, check viewer success, then try another browser.
4. Use the downloaded gds_render PNG while investigating.

CHECK Identify deployment or rendering first, then use the matching remedy.

Step 15: deliberately start a fresh complete run



Actions → choose gds in the workflow list. • Select Run workflow; choose the intended branch. • Select Run workflow in that menu. • Check the new run uses your intended saved commit.

Official Tiny Tapeout FAQ image: Run workflow controls in an older example repository.

CHECK The fresh run belongs to the branch and complete saved version you intended.

Step 16: optional local setup in Ubuntu or WSL

```
sudo apt-get update
sudo apt-get install -y git python3 python3-venv
sudo apt-get install -y python3-pip make iverilog
git clone https://github.com/YOUR_USERNAME/YOUR_REPOSITORY.git
cd YOUR_REPOSITORY
git rev-parse HEAD
```

Use an Ubuntu/WSL terminal; these are Linux commands. • On your own machine install prerequisites below; managed labs use supplied tools. • Replace the URL placeholders, clone your project and enter its root.

CHECK The project root contains info.yaml, src and test; record the checked-out commit.

Step 16: run RTL, then explicitly check XML

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r test/requirements.txt
cd test
make -B
python -m cocotb_tools.check_results results.xml
```

1. From the project root, create and activate the Python environment.
2. Install the template test requirements; run inside test.
3. Keep results.xml and tb.fst; use Actions gl_test for the first mapped-cell result.

CHECK Inside test: results.xml and tb.fst exist; the explicit checker succeeds.

Step 17: specify one small extension first

1. Example: an enabled count-down counter, with 0 wrapping to 255.
2. Define reset priority, hold and the new terminal flag.
3. Keep the fixed Tiny Tapeout interface.
4. Predict boundary cases before editing RTL.

WORKED EXAMPLE

Proposed count-down boundary: count 0, enable 1, reset released → next count 255.

CHECK Your written rules specify both the stored count and the combinational flag.

Step 17: update, commit and recheck the extension

1. Update RTL, Python model/assertions, metadata and datasheet together.
2. New files need both source lists; renamed tops need all three locations.
3. Commit; check test, docs and all four gds jobs for that version.
4. Compare its own cell counts and utilisation.

CHECK **The extension has its own matching source, tests, documents and layout evidence.**

Step 18: archive one checked revision

1. Record repository, branch, full commit and the three run links.
2. Keep datasheet, XML, FST, layout/netlist, metrics and build logs.
3. Keep precheck reports and source/provenance manifests.
4. Label original-counter and extension evidence separately.

CHECK Another person can identify the version and reopen its original evidence.

Step 18: stop before fabrication actions

1. The free result is a tested, checked and archived layout.
2. Fabrication needs a separate approved order and submitted revision.
3. A later GitHub commit does not automatically replace a submitted revision.
4. Stop before payment, ordering or fabrication submission.

CHECK You can explain the completed design result and the separate fabrication boundary.

Follow one design through changing representations

RTL

Describes state and logic

Netlist

Names cells and connections

Layout

Places cells and wires

Reports

Check the implementation

The counter's specified behaviour stays the same. • Each stage produces a different kind of evidence.

CHECK Which artifact contains geometry rather than behaviour?

Synthesis starts from a precise update rule

```
always @(posedge clk) begin
  if (rst)
    count <= {WIDTH{1'b0}};
  else if (en)
    count <= count + 1'b1;
end
assign wrap = en & (&count);
```

1. Eight count bits need stored state.
2. Reset and enable decide the next stored value.
3. The terminal-count expression becomes combinational logic.

CHECK At count 255 with enable low: count holds; wrap is low.

The synthesis result reports 47 cells

1. The reference synthesis report contains 47 cells.
2. Its netlist names instances and the wires connecting them.
3. Placement, clock distribution and routing still follow.

WORKED EXAMPLE

47 synthesis cells ≠ 4,600 final physical instances.

CHECK Find the synthesis summary in GDS_logs; record its commit.

The PDK connects the tools to a manufacturing process

1. A PDK supplies process layers, rules, models and technology data.
2. A standard-cell library supplies reusable digital circuits.
3. This template uses SKY130 and its high-density cell library.

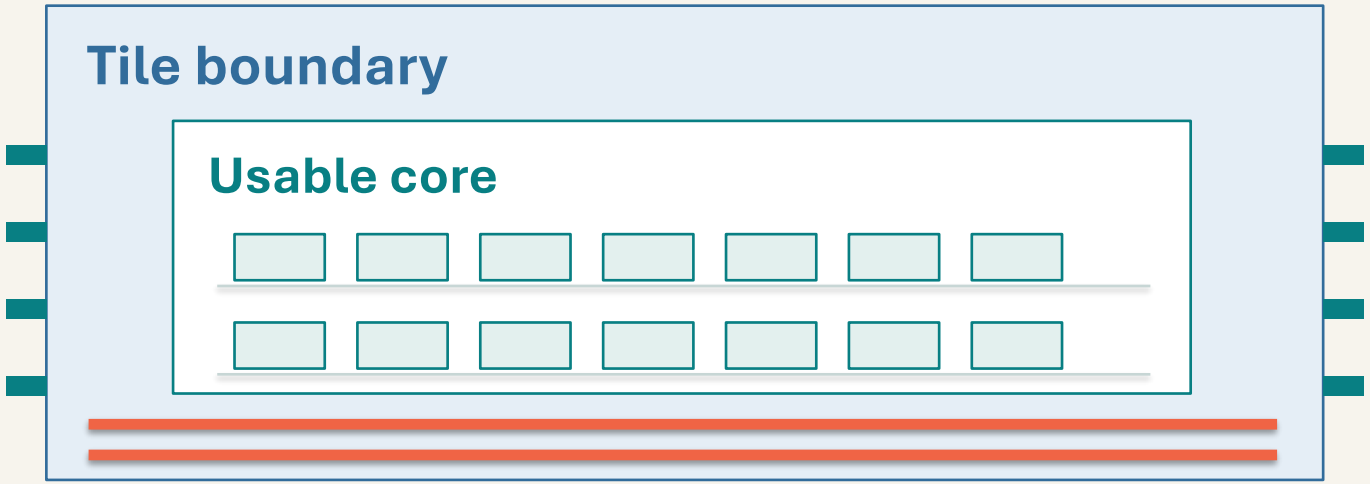
WORKED EXAMPLE

Changing a filename cannot turn SKY130 geometry into IHP geometry.

CHECK Record sky130A, the library and the PDK/tool versions.

Floorplanning establishes the space and interfaces

Floorplan = boundary + usable core + pins + power plan.
Tiny Tapeout provides a fixed tile within a larger shared chip.



- Boundary
  Limits the tile
- Core / rows
  Legal cell positions
- Pins
  External connections
- Power rails
  Distribute supply

Conceptual diagram — not measured geometry

CHECK A tile is one part of a shared chip.

Placement must leave room for connections

1. Place cell instances in legal rows and positions.
2. Keep connected cells sensibly arranged.
3. Cell area, wiring space and timing all affect the result.

WORKED EXAMPLE

A small netlist can still be awkward to place or route.

CHECK Read a placement failure before changing density or tile size.

Calculate what 4.96131% utilisation means

1. Reported core utilisation: 4.96131%.
2. Counted area includes physical support cells such as taps.
3. It is not a percentage of the entire shared chip.

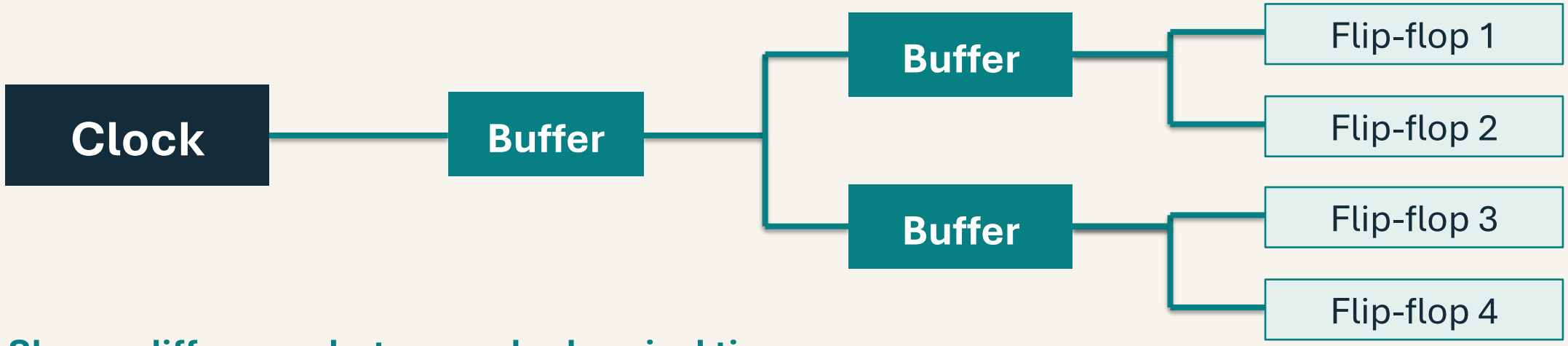
WORKED EXAMPLE

$$818.285\ \mu m^2 \div 16{,}493.3\ \mu m^2 \times 100 \approx 4.96131\%$$

CHECK **The remaining 95% is not guaranteed capacity for any extension.**

A clock tree distributes the sampling event

One clock must reach many storage elements.
Buffers distribute load; physical clock arrivals can differ.



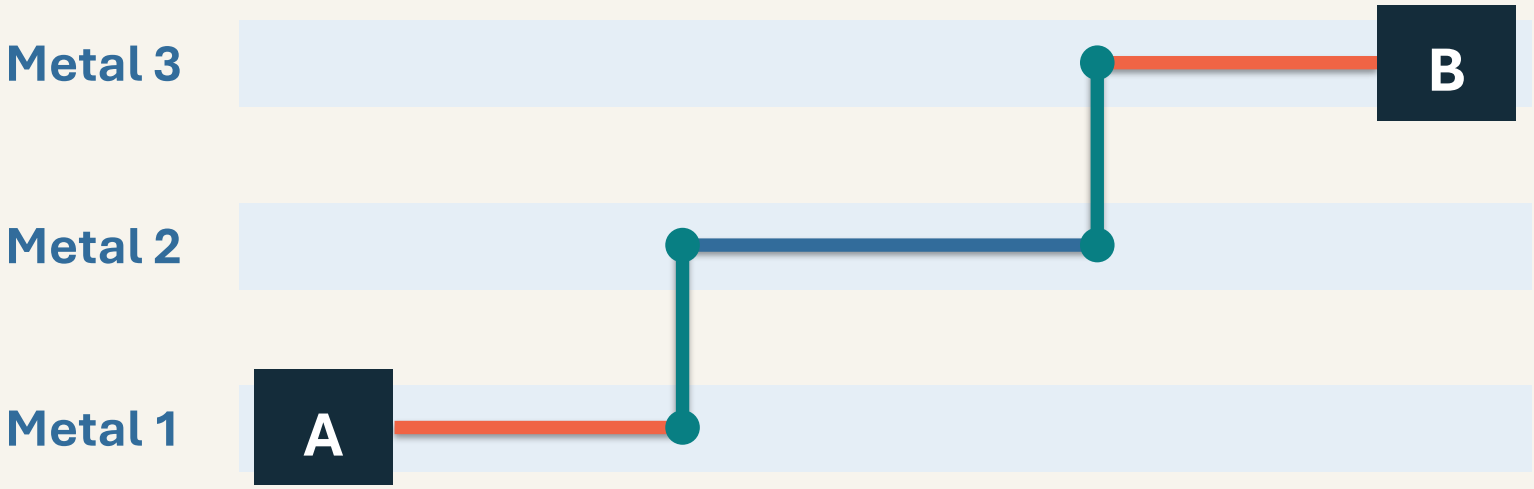
Skew = difference between clock arrival times.

Conceptual branching tree — not the reference clock network

CHECK RTL's single clock name does not imply zero physical skew.

Routing turns netlist connections into wires

Routing makes physical wire paths; vias change layers.
Width, spacing and required connectivity must be checked.



Conceptual layer view — spacing and wire lengths are not measured

Metal wires run on a layer.

Vias connect different layers.

One required net

CHECK Reference: 16 unused, non-critical disconnected inputs; zero critical.

Extraction estimates the electrical effects of wires

1. Real wires have resistance and capacitance.
2. Parasitic extraction estimates them from implemented geometry.
3. Timing analysis uses this information.

WORKED EXAMPLE

Changing a route can change delay while preserving connectivity.

CHECK Antenna checking addresses fabrication charge, not radio reception.

A 50 MHz target means a 20 ns clock period

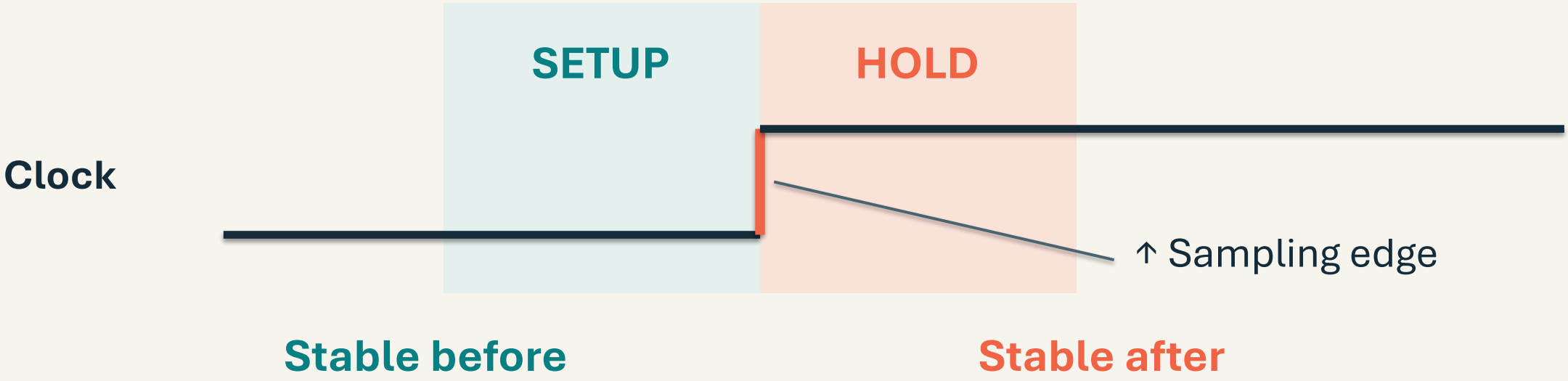
```
period = 1 / frequency
       = 1 / 50,000,000 seconds
       = 20 nanoseconds
```

1. The reference uses a 20 ns clock constraint.
2. Its generic input and output delays are 4 ns.
3. These describe the analysed interface, not measurements of a board.

CHECK Read the SDC constraints before interpreting timing slack.

Setup and hold protect opposite sides of an edge

Data must be stable around the receiving clock edge.
Setup is before; hold is after. STA checks both.



Conceptual timing windows — widths do not represent cell requirements

CHECK A passing setup check alone does not establish a passing hold check.

Read the real timing result with its conditions

1. Worst reported setup slack: +8.64285 ns.
2. Worst reported hold slack: +0.109636 ns.
3. Reported setup and hold violation counts: zero.

WORKED EXAMPLE

Positive margins support the stated target under the analysed conditions.

CHECK They do not measure a fabricated chip's maximum frequency.

Practise interpreting a timing failure

TRY IT FIRST

Hypothetical teaching example; not a failure in the verified run.

Hypothetical setup slack is −0.8 ns. What does it mean, and what should you inspect first?

WORKED ANSWER

The path is 0.8 ns late under its stated constraints. Inspect the path and constraints.

CHECK Would slowing the clock repair an incorrect count sequence? No.

DRC checks geometry against process rules

1. Design rule checking tests permitted widths, spacing and other geometry.
2. Correct logic can still have illegal physical shapes.
3. The reference reports zero Magic DRC errors.

WORKED EXAMPLE

Illustrative error: two unrelated wires placed closer than allowed.

CHECK State which checks ran and passed.

LVS checks whether the geometry represents the circuit

DRC asks whether the shapes follow checked rules.

LVS compares extracted connectivity with the intended netlist.

The reference Netgen result matches; reported LVS errors are zero.

A legally spaced wire connected to the wrong node can fail LVS.

CHECK Passing DRC does not by itself establish passing LVS.

Check scope: KLayout ran in precheck

Hardening: Magic DRC and Netgen LVS ran.

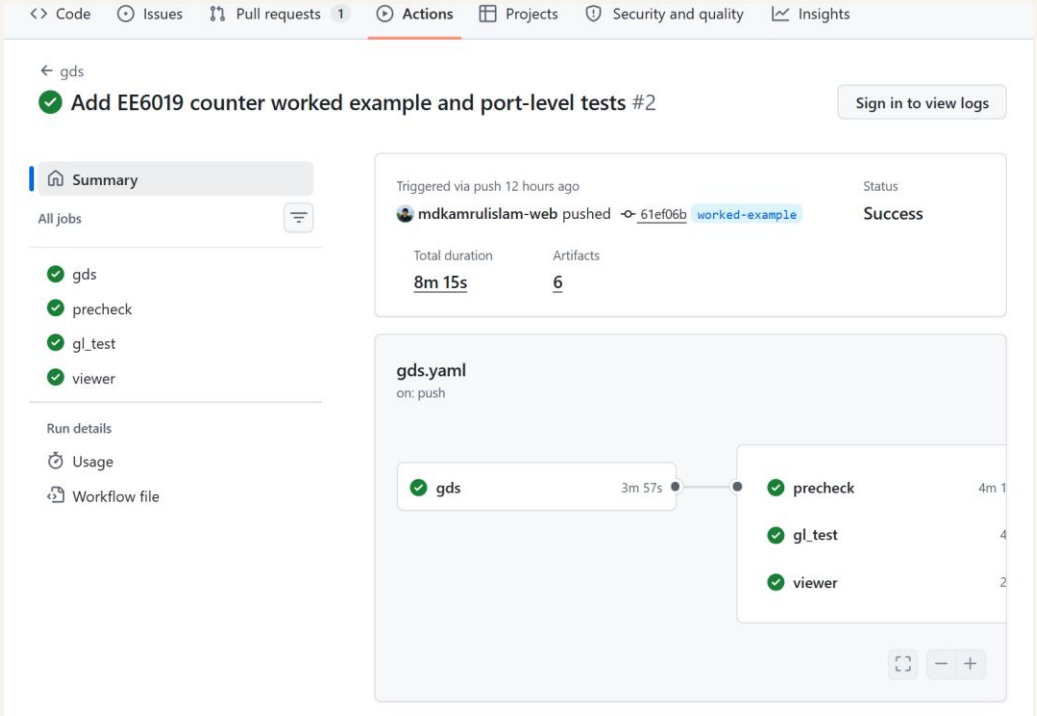
Hardening's KLayout DRC and XOR steps were disabled.

Separate precheck ran KLayout geometry checks, including FEOL and BEOL.

Disabled in one stage ≠ absent from every check.

CHECK FEOL: device-side rules; BEOL: interconnect-side rules.

Precheck verifies the package for integration



1. The reference precheck passed all 15 reported checks.
2. It checks integration requirements, pins, boundaries and geometry.
3. Require gds, precheck, gl_test and viewer for the same commit.

Actual reference run: four successful jobs at 61ef06b.

CHECK A successful viewer does not prove its parallel jobs passed.

The gate-level test checks implemented behaviour

RTL test: exercises the behavioural hardware description.

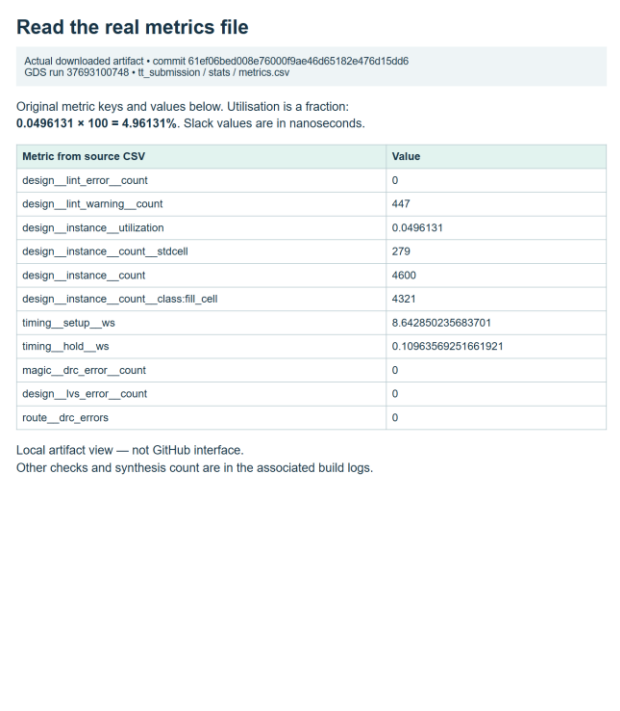
Gate-level test: exercises the implementation netlist with cell models.

This template uses functional/unit-delay settings, without extracted SDF.

Both tests check top-level pins, so renamed internal signals are unnecessary.

CHECK Gate-level functional simulation complements STA; it does not replace it.

Three cell counts describe different things



Read the real metrics file

Actual downloaded artifact • commit 61ef06bed008e76000f9ae46d65182e476d15dd6
GDS run 37693100748 • tt_submission / stats / metrics.csv

Original metric keys and values below. Utilisation is a fraction:
0.0496131 × 100 = 4.96131%. Slack values are in nanoseconds.

Metric from source CSV	Value
design__lint_error__count	0
design__lint_warning__count	447
design__instance__utilization	0.0496131
design__instance__count__stdcell	279
design__instance__count	4600
design__instance__count__class:fill_cell	4321
timing__setup__ws	8.642850235683701
timing__hold__ws	0.10963569251661921
magic__drc_error__count	0
design__lvs_error__count	0
route__drc_errors	0

Local artifact view — not GitHub interface.
Other checks and synthesis count are in the associated build logs.

Actual downloaded reference metrics, opened locally.

1. 47 cells: synthesis result.
2. 279 non-filler instances: includes 225 taps.
3. 4,600 final instances: includes 4,321 fillers.

A passing run can still contain warnings

1. Reference lint errors: zero.
2. Reference timescale warnings: 447.
3. Classify a warning by its cause and effect before changing code.

WORKED EXAMPLE

447 warnings are not 447 failed cocotb assertions.

CHECK Record the warning and the successful result together.

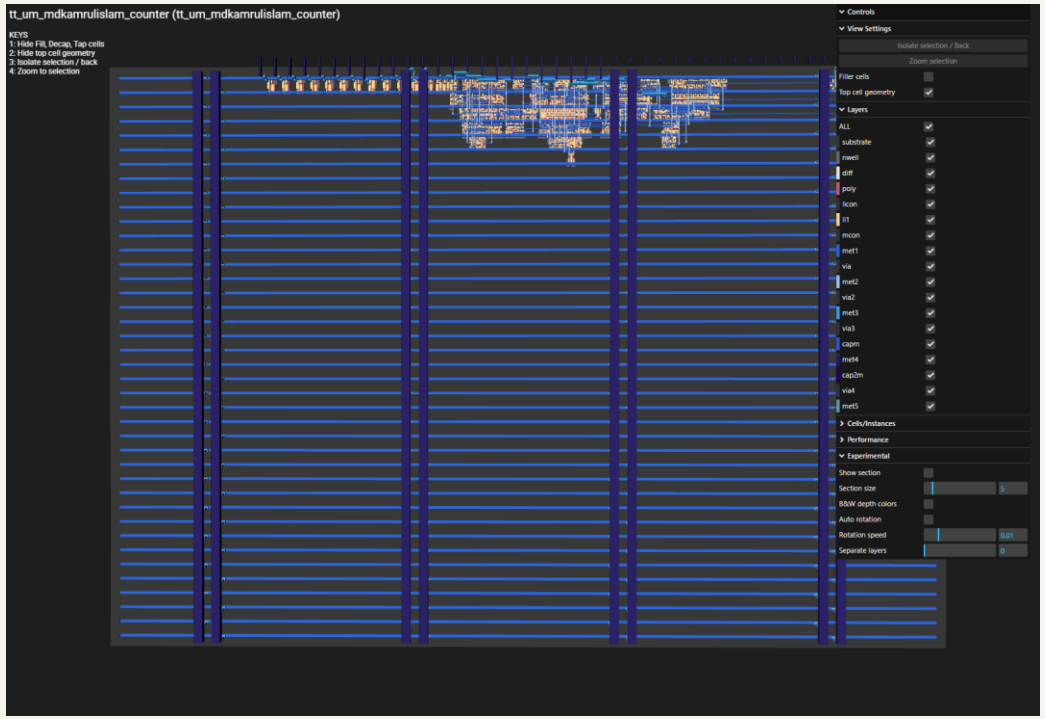
Archive files that support your conclusion

```
evidence/
  identity.txt
  rtl-test/results.xml + tb.fst
  layout/
  build-logs/
  datasheet/
  observations.txt
```

1. Download artifacts while they remain available.
2. GDS/OAS stores geometry. The netlist lists connections.
3. The artifact is a file package. Ordering is a separate action.

CHECK Record repository, full commit, run URLs and versions.

Explore the real layout without overstating the picture



Actual counter viewer, with fill/decap/tap hidden.

1. Coloured shapes represent manufacturing layers.
2. The native toggle hides filler, decap and tap cells together.
3. Rotation and zoom help locate cells and routed connections.

CHECK Geometry exists; reports still establish timing and rule results.

Finish the free exercise with one checked revision

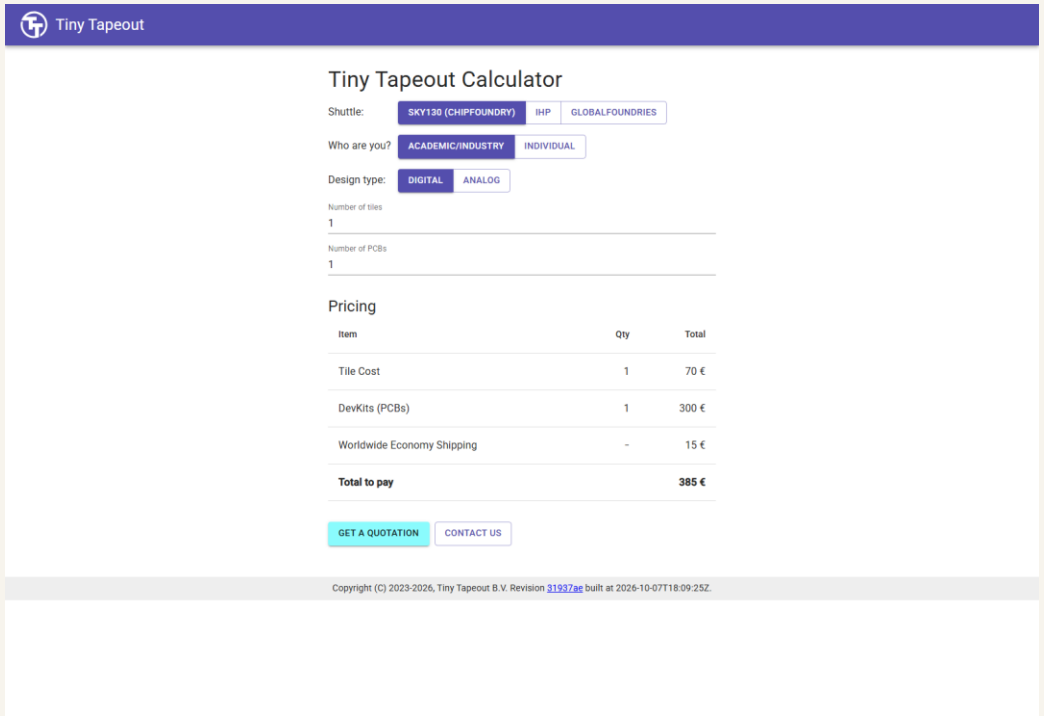
1. Keep specification, source and pin mapping.
2. Match RTL, docs and all four GDS jobs to one commit.
3. Archive artifacts, waveform observations, metrics and limitations.

WORKED EXAMPLE

Explain reset, hold, wrap and one timing result to a demonstrator.

CHECK You can complete this package without buying silicon.

A tile and a development kit are separate purchases

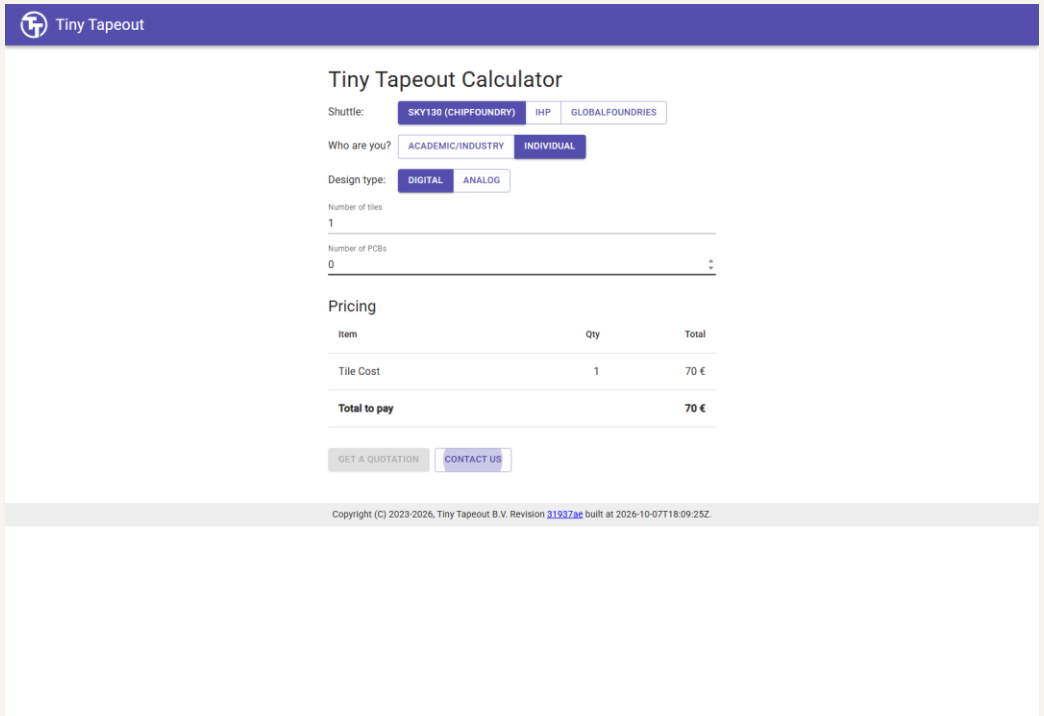


1. 8 October 2026 calculator: SKY130, academic/industry.
2. One digital tile €70; one kit €300; shipping €15.
3. Displayed total: €385.

Genuine dated public estimate; no quote or order was requested.

CHECK Check current terms before any approved order.

A €70 tile selection includes no development kit



1. 8 October 2026 calculator: individual, one tile, zero PCBs.
2. The displayed total was €70.
3. A tile credit does not automatically include hardware.

Actual public tile-only selection; no order was placed.

CHECK Ask what is included, rather than comparing totals alone.

Understand the later order and submission sequence

Reviewed files	Project/order	Payment or zero total	Submit revision
Passing builds	Sign in; select repository	Complete order stage	Choose the source version

Owner approval covers shuttle, budget, rights and delivery. • A zero-total order is still an order. • This explains a later process; the exercise stops earlier.

CHECK No authenticated payment screens were visited for this lesson.

A new GitHub commit does not replace a submitted revision

GitHub commit: saves a version and can trigger builds.

Order stage: books the selected allocation.

Submit a new revision: explicitly selects design source for integration.

After an RTL change: new passing builds → deliberate new submission.

CHECK Record the commit accepted by the submission before the deadline.

Integration creates the shared chip before tapeout

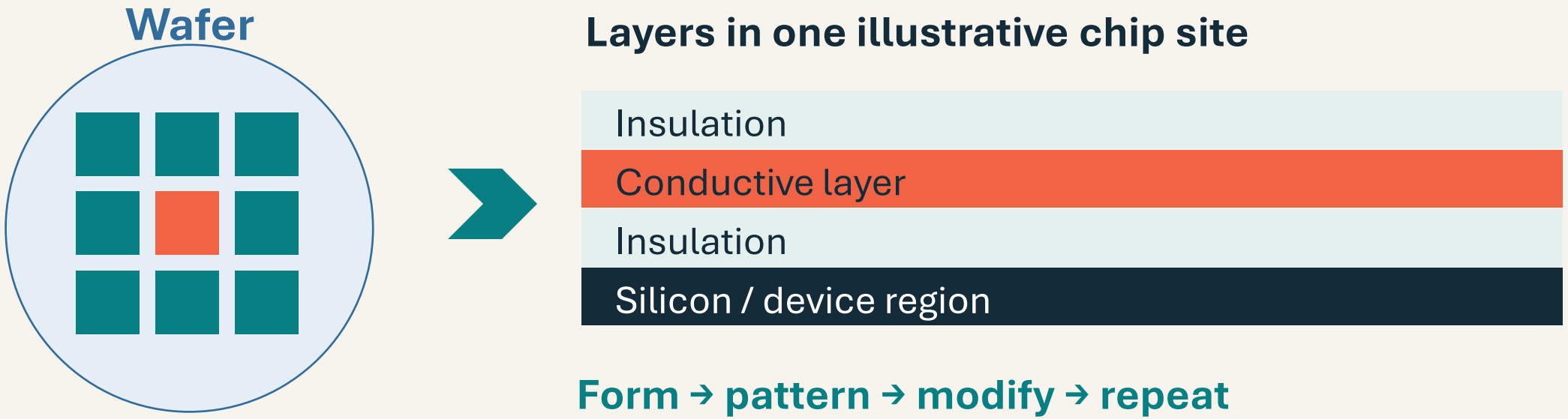
Checked tile	Shared chip	Final checks	Tapeout
One user design	Many designs and infrastructure	Integrated implementation	Manufacturing-data handover

Accepted tiles join shared selection and chip infrastructure. • The complete chip needs additional integration checks. • Tapeout hands over final manufacturing data.

CHECK Passing one standalone tile is only part of a complete chip.

Masks and repeated process steps create wafer structures

A wafer contains repeated chip sites.
Patterns and repeated process steps form devices and interconnect.



Form → pattern → modify → repeat

Conceptual wafer and layers — not a SKY130 cross-section or mask count

CHECK The fab uses manufacturing data, not a screenshot of the viewer.

Transistor networks implement gates and stored state

Transistors

Physical devices

Library cells

Reusable circuits

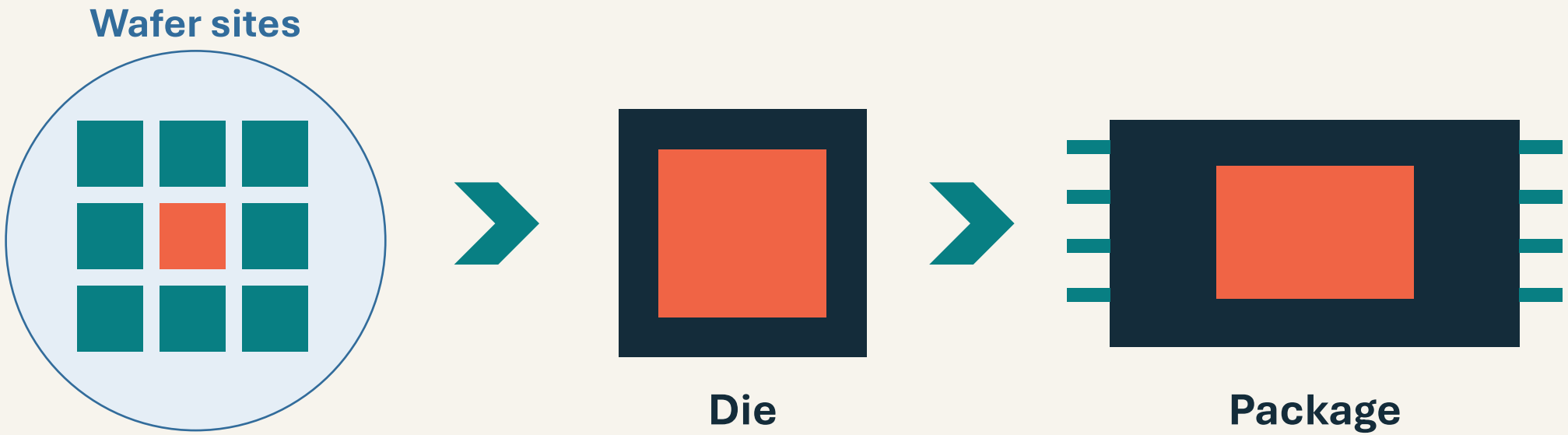
Counter netlist

Connected instances

Transistors help control conducting paths. • CMOS uses complementary networks to implement logic. • The standard-cell library supplies the counter's gates and flip-flops. • An inverter maps logical 0 to 1, and 1 to 0.

Testing and assembly continue after wafer processing

Test devices; dice the wafer into dies; package for protection and connections.
Board assembly and testing prepare usable hardware.



Conceptual separation and packaging — actual delivery depends on the service

CHECK Delivery depends on the selected service and kit; check current schedules.

Read the counter as binary on a board

1. Select the design and use a controlled learning clock.
2. Hold rst_n low across a rising edge; release it, then enable ui_in[0].
3. Read uo_out[7:0]; uio[0] flags enabled count 255.

WORKED EXAMPLE

Count 5 = 00000101: output bits 2 and 0 are high.

CHECK The RTL does not include a decimal-display decoder.

A physical button is not an ideal testbench input

Synchronisation reduces risks from asynchronous sampling.

Debouncing suppresses repeated changes from switch contacts.

These solve different problems; this counter assumes synchronous enable.

One press can produce several electrical transitions near clock edges.

CHECK A simulator's logical 1 does not mean one volt.

Local simulation is an optional faster feedback loop

Browser route: Actions supplies the simulation and physical tools.

Local RTL route: you supply Linux, Python, make and Icarus.

Use Ubuntu or a managed lab environment; these are not PowerShell commands.

CHECK Installing a full physical-design suite is unnecessary for this RTL exercise.

Check tools and obtain your saved source

```
sudo apt-get update
sudo apt-get install -y git python3 \
  python3-venv python3-pip make iverilog
git --version
python3 --version
make --version
iverilog -V
```

1. On your own Ubuntu installation, install missing prerequisites.
2. Clone your repository or extract its complete project ZIP.
3. Test the same source version as your saved reference run.

CHECK Project root contains info.yaml, src and test.

Install test packages inside an isolated environment

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r test/requirements.txt
```

1. Run from the project root.
2. A virtual environment isolates the test's Python packages.
3. Reference requirements pin cocotb 2.1.0 and pytest 8.4.2.

CHECK Python packages do not install the Icarus Verilog executable.

Run the test, then explicitly check its XML result

```
cd test
make -B
python -m cocotb_tools.check_results results.xml
```

1. Icarus compiles the design; cocotb drives inputs and checks assertions.
2. The XML checker verifies reported failures and errors.
3. Reference result: one passing test; 11,368 ns simulated time.

CHECK Require results.xml and tb.fst; make alone can conceal a test failure.

Local gate-level testing needs matching implementation files

```
export PDK_ROOT=/path/to/installed/pdk
SC="$PDK_ROOT/sky130A/libs.ref"
SC="$SC/sky130_fd_sc_hd/verilog"
test -f "$SC/primitives.v"
test -f "$SC/sky130_fd_sc_hd.v"
test -f gate_level_netlist.v
make -B GATES=yes
python -m cocotb_tools.check_results results.xml
```

1. Copy the exact implementation netlist to test/gate_level_netlist.v.
2. PDK_ROOT names the directory containing sky130A.
3. These checks inspect existing files; they do not install a PDK.

CHECK Correct missing paths before make; use the build's pdk.json identity.

Inspect the waveform and diagnose one cause at a time

Missing tool	Missing source	Import error	Wrong count
Check installation	Check lists and directory	Check environment	Compare specification and wave

Open tb.fst in web Surfer or an available native viewer. • Inspect clock, reset, enable, count and terminal-count output. • Read the first useful error before changing anything.

CHECK Local changes need a saved commit and fresh Actions evidence.

Exercise 1: Binary output

1. Count is decimal 37. Write its eight-bit output and list the high pins.

WORKED EXAMPLE

Predict it first.
Then advance for the worked answer.

Answer 1: Binary output

1. 37 = 32 + 4 + 1, so uo_out = 00100101. Bits 5, 2 and 0 are high. No decoder is needed to read those bits as a number in software; a human-facing decimal display would need decoding.

Exercise 2: The enable bit

1. Reset is inactive. Write ui_in=10000000, then apply two rising edges. The starting count is 9. What happens? What changes if ui_in=10000001 instead?

WORKED EXAMPLE

Predict it first.
Then advance for the worked answer.

Answer 2: The enable bit

1. In the first case bit 0 is zero, so count stays 9 at both edges. In the second case bit 0 is one, so the counts after the two edges are 10 and 11. Bits 7:1 do not affect this example.

Exercise 3: Reset priority

1. Starting count is 18. Before the next rising edge set rst_n=0 and enable=1. What should count and wrap be after that edge?

WORKED EXAMPLE

Predict it first.
Then advance for the worked answer.

Answer 3: Reset priority

1. Count is 0 because reset has priority. Wrap is 0 because the settled count is not 255. This is directly exercised by step(enable=1, reset=True).

Exercise 4: Reset between edges

1. Count is 12 and enable is zero. Reset becomes active halfway between two rising edges. Does count become zero at the moment reset changes?

WORKED EXAMPLE

Predict it first.
Then advance for the worked answer.

Answer 4: Reset between edges

1. No. It remains 12 until the next rising edge, then becomes zero. Active-low polarity does not make a reset asynchronous.

Exercise 5: Terminal count

1. Starting count is 254, reset is inactive and enable stays high. Predict count and wrap after three rising edges.

WORKED EXAMPLE

Predict it first.
Then advance for the worked answer.

Answer 5: Terminal count

1. The pairs (count, wrap) are (255,1), (0,0) and (1,0). The flag marks the 255 state, not the zero state after overflow.

Exercise 6: Hold at terminal count

1. Count is 255 and enable becomes zero. Predict the next two edge samples. Then enable becomes one before a third edge; predict that edge sample too.

WORKED EXAMPLE

Predict it first.
Then advance for the worked answer.

Answer 6: Hold at terminal count

1. The first two samples are count 255 and wrap 0. After re-enabling but before the third edge, wrap is high while count is still 255. After the third edge, count is 0 and wrap is 0.

Exercise 7: Width masking

1. The Python model is 255, reset is false and enable is 1. Calculate (model + enable) & 0xFF by hand.

WORKED EXAMPLE

Predict it first.
Then advance for the worked answer.

Answer 7: Width masking

1. The sum is 256, hexadecimal 0x100. Masking with 0x0FF retains the lower eight bits: 0x100 & 0x0FF = 0x000. The expected count is zero.

Exercise 8: Interpret uio direction

1. The wrapper drives uio_oe=00000001. Which uio pins are outputs? Does uio_out[7:1]=0 mean the design drives those physical pins low?

WORKED EXAMPLE

Predict it first.
Then advance for the worked answer.

Answer 8: Interpret uio direction

1. Only uio[0] is enabled as an output. Pins 7:1 have disabled output drivers and remain inputs. Their output-data values are zero, but disabled drivers do not force the external pins low.

Exercise 9: Diagnose a hypothetical mismatch

1. Imagine your modified test reports expected count 1 but observed count 0 after an enabled edge. List four things to inspect before altering the assertion.

WORKED EXAMPLE

Predict it first.
Then advance for the worked answer.

Answer 9: Diagnose a hypothetical mismatch

1. Check whether reset was still active; whether the code set ui_in[0] rather than another bit; whether the simulator compiled the intended wrapper/core files and top-module name; and whether observation waited for the appropriate settled output.
2. Also compare the expected model with the written specification. This is a hypothetical diagnostic exercise; the reviewed worked-example test passed.

Exercise 10: Modify the counter to count down

1. Write the expected sequence from reset for three enabled rising edges before changing the RTL. Define what the flag should mean in a count-down design.

WORKED EXAMPLE

Predict it first.
Then advance for the worked answer.

Answer 10: Modify the counter to count down

1. With unsigned 8-bit arithmetic the sequence is 255, 254, 253 after the three edges: 0−1 wraps to 255.
2. A reasonable terminal/underflow flag is high when count is zero and enable is high, before the next decrement wraps to 255.
3. Specify that rule explicitly; do not retain the count-up rule “all bits high” merely because it compiles.

Answer 10: Modify the counter to count down (continued)

1. Update the model to subtract enable modulo 256, update the flag expectation and its boundary tests, then compare results.

2. Any modified design is a new version requiring new evidence; the public passes for the original count-up revision do not certify it.

Exercise 11: Test a specification change

1. Suppose the new requirement is “generate a registered pulse after overflow.” Why is renaming wrap insufficient?

WORKED EXAMPLE

Predict it first.
Then advance for the worked answer.

Answer 11: Test a specification change

1. A name does not change the expression. The existing continuous assignment detects the current terminal state.
2. A registered post-overflow pulse needs state and an edge-update rule, including what reset and disable do to it.
3. Write that specification and test the pulse relative to count 255→0 before implementing it.

Exercise 12: Judge evidence

1. You have a green RTL test, a layout viewer screenshot and no timing report. Can you conclude the design meets its target clock frequency? Can you conclude a chip has been ordered?

WORKED EXAMPLE

Predict it first.
Then advance for the worked answer.

Answer 12: Judge evidence

1. Neither conclusion follows. Functional simulation checks sampled behaviour, and the viewer shows geometry. Timing reports check the implementation against the clock constraints.

2. A chip order additionally requires an explicit authorised shuttle booking/payment/submission process. Keep those evidence categories separate.

Explain the result before calling it finished

1. Predict reset, count, hold, overflow and the pre-overflow wrap flag.
2. Show matching source, test, documentation and layout evidence.
3. Distinguish simulation, timing analysis, layout checks and silicon tests.
4. Archive the checked revision before considering a fabrication order.

CHECK Demonstrate the circuit and explain what each form of evidence proves.

Choose a small next design change

1. Count down: specify whether zero wraps to 255.
2. Parallel load: specify its pins and reset/load/enable priority.
3. Update RTL, the independent model, metadata and datasheet together.
4. Inspect the new commit’s tests, checks and cell measurements.

WORKED EXAMPLE

Change the specification first. Then change the implementation and its checks.

CHECK Old screenshots do not prove a modified circuit works.

Keep the learning package together

1. PowerPoint: editable diagrams, notes and linked sources.
2. Course PDF: stable slide layout for reading and printing.
3. Word/PDF manual: deeper explanations and the complete source appendix.
4. Starter ZIP and original artifacts: exact source bytes and real evidence.

CHECK Use the full package when teaching or reproducing the lesson.